

INDEPENDENT MATHEMATICAL RESEARCH

Semantic Extension Profiles and the Lower-Bound Frontier for P versus NP

Verification, search, semantic extension, monotone representations, circuit complexity, constructive refutation, lifting, and lower bounds

Pearl Bipin Pulickal

Independent Researcher

September 2026

Self-contained research monograph

Scope of the volume

This book does not claim a proof of $P \neq NP$. Its central mathematical object is the semantic extension profile of a Boolean instance: the function recording, for every partial assignment, whether a satisfying completion remains. The underlying extension operation is related to classical monotone-extension and SAT-representation machinery; the manuscript therefore does not claim that the raw operation itself is new. The principal constructive contribution of the present revision is a unified, audited framework that connects this semantic object to uniform circuit realization, restricted lower-bound laboratories, constructive refutation, meta-complexity, lifting, and proof complexity. For the universal SAT family, P-uniform polynomial circuit realizability is proved equivalent to $SAT \in P$. The decisive superpolynomial lower bound remains open.

The document is intentionally generous in exposition. Earlier valid mathematics is retained, failed routes are preserved when they teach a precise lesson, and each new research object is tested against the quantifiers and representation changes that caused earlier proposals to fail.

Preface

This volume presents an independently audited research programme. Earlier motivational claims are retained only where the underlying mathematics survives explicit logical and adversarial checks.

The principal constructive development is the semantic extension-profile framework. Rather than assigning hardness to one input or one trace, we associate to an instance the finite function that records satisfiability after every partial assignment. This operation is calibrated against the established literature on monotone extension, SAT representations, and partial-assignment semantics. The manuscript then separates three logically different layers: the semantic object, a uniform realization of that object, and the lower-bound engine used to attack such realizations.

For a universal SAT family, a P-uniform polynomial realization is shown to be exactly equivalent to SAT being in P. The remaining problem is therefore not hidden but exposed: one needs a theorem excluding such uniform realizations, or a stronger circuit lower bound, by a mechanism that survives representation changes and the known lower-bound barriers.

Author. Pearl Bipin Pulickal — Independent Researcher.

Scope. The book contains proved results, conditional implications, explicitly labeled conjectural or exploratory directions, and adversarial audits. It does not claim a solution of the P versus NP problem.

Reading Guide and Status Legend

The first part of the book builds the standard verification/search formalism. The middle part is an adversarial audit of finite evidence, diagonalization, resource invariants, communication, proof complexity, and known lower-bound barriers. The semantic residual chapters introduce the principal new object and derive its exact relationship with SAT. The appendices then collect the status ledger and finite audit checklist.

Theorem — proved from the stated assumptions.

Proposition / Lemma — proved local consequence.

Conditional theorem — implication proved; hypothesis open or separately stated.

Conjecture — explicit unproved mathematical statement.

Research direction — proposed route, not a truth claim.

Heuristic — explanatory intuition, never promoted to proof.

Experiment — finite observation, never promoted to an asymptotic theorem.

Notation

We use $\Sigma = \{0, 1\}$, $|x|$ for length, $\mathbf{P}, \mathbf{NP}$ for deterministic and nondeterministic polynomial time, $\mathbf{SAT}$ for satisfiability, and $\mathbf{CC}(f)$ for general Boolean circuit size. Partial assignments use $\mathbb{T} = \{0, 1, \star\}$, with $\star$ denoting an unassigned coordinate. Uniform circuit families are required to be constructible by a deterministic polynomial-time constructor.

Contents

Preface	i
Reading Guide and Status Legend	ii
Notation	iii
1 Problem, Scope, and Logical Standards	1
1.1 The Problem and the Standard of the Attempt	1
1.1.1 Why the gap is mathematically non-trivial	1
1.1.2 A methodological rule	2
1.2 Formal Preliminaries	3
1.2.1 Polynomial time	3
1.2.2 Search relations	3
2 Verification, Search, and Self-Reduction	5
2.1 What the Verification–Generation Intuition Actually Says	5
2.2 Search-to-Decision for Polynomially Balanced Self-Reducible Relations . . .	6
2.3 The Exact Search-to-Decision Equivalence for SAT	6
2.3.1 Restricted prefix formulas	7
3 Auditing, Finite Evidence, and False Shortcuts	8
3.1 A Precise Formalization of the “Audit Team” Intuition	8
3.1.1 Error detection versus universal certification	8
3.2 Finite Ensembles of Auditors	9
3.3 Finite Evidence, Adversarial Search, and the Limits of Testing	9
3.3.1 A stronger adversarial observation	10
3.4 Gödel Incompleteness: Relevant Warning, Invalid Shortcut	10
3.5 The Epistemic Regress and the Meaning of “Proof”	11
4 The Quantifier Gap	12
4.1 The Quantifier Ledger	12
4.2 What Would a Direct Proof of $P \neq NP$ Have to Establish?	13
4.2.1 A proof-by-diagonalization template	13

5	From Intuition to a Lower-Bound Program	14
5.1	The First Strong Candidate: A Verification-to-Generation Gap Axiom . . .	14
5.1.1	A more structured target	14
5.2	Barrier I: Relativization	15
5.3	Barrier II: Natural Proofs	15
5.4	Barrier III: Algebrization	15
5.5	What the Barriers Tell Us About the Proposed Program	16
6	Resource Transfer, Proof Complexity, and Structural Routes	17
6.1	A More Productive Route: Search, Proof Systems, and Witness Structure .	17
6.1.1	Why restricted proof-system lower bounds still matter	17
6.2	A Formal “Creator–Auditor” Model	18
6.3	A Conditional Theorem That Captures the Intended Logic	19
6.4	Attempted Proof of the Universal Generator Lower Bound	19
6.4.1	Step 1: enumerate polynomial-time algorithms	19
6.4.2	Step 2: the standard diagonal instinct	20
6.4.3	Step 3: try to encode the diagonal computation into the witness . .	20
6.4.4	Step 4: why the obvious self-reference does not finish the proof . .	20
6.4.5	Step 5: the relativization test	21
6.4.6	Step 6: a repaired diagonal target	21
6.4.7	The revised missing lemma	21
6.5	From Algorithm-Relative Certificates to Resource Transfer	22
6.5.1	Resource-transfer certificates	22
6.5.2	Why algorithm-relative is necessary but not sufficient	23
7	Adversarial Attacks on Candidate Resources	24
7.1	Kolmogorov and Circuit Complexity: Two Reviewer Suggestions Under Direct Attack	24
7.2	Direct Attack on the Dynamic Communication-Trace Proposal	25
7.2.1	A standard communication simulation is a genuine transfer theorem	25
7.2.2	The communication ceiling	25
7.2.3	The trace-length dichotomy	26
7.2.4	Why a fixed partition is vulnerable to representation changes . . .	26
7.2.5	Dynamic traces and pebbling	27
7.2.6	Current verdict on the dynamic communication trace	27
7.3	Meta-Complexity and Hardness Magnification as Candidate Bridges	28
7.3.1	A residual meta-complexity problem	28
7.3.2	Gate elimination as a general-circuit lower-bound laboratory	29
7.4	A More Precise Meaning of Accessibility	29
7.5	The Universal Extraction Lemma	29

7.6	Adversarial Research Ladder	30
7.7	Why Brute-Force Explosion Is Not Yet a Lower Bound	30
7.8	Cryptography: Powerful Evidence, Not a P versus NP Proof	31
7.9	The “If It Were P, Someone Would Have Found It” Argument	31
8	The Repaired Research Program	32
8.1	A Constructive Search-to-Decision Framework	32
8.2	A Stronger Adversarial Program	32
8.2.1	Question A: Can we lower-bound generators in a restricted model?	32
8.2.2	Question B: Can the restricted model approximate general polynomial time?	33
8.2.3	Question C: Can an algorithm-to-lower-bound conversion be proved?	33
8.3	An Explicit Audit of Common Failed Proof Patterns	33
8.3.1	Pattern 1: “There are exponentially many possibilities.”	33
8.3.2	Pattern 2: “Verification is polynomial, so search is harder.”	33
8.3.3	Pattern 3: “Many AIs cannot find a proof.”	33
8.3.4	Pattern 4: “Gödel prevents the system from proving itself.”	33
8.3.5	Pattern 5: “No one has found the shortcut.”	34
8.3.6	Pattern 6: “Peer review eventually accepts the proof.”	34
9	Hard-Coding, Reduction Stability, and Restricted Models	35
9.1	The Central Attack: The Original Universal Γ Cannot Exist	35
9.1.1	The hard-coding theorem	35
9.1.2	A stronger version: finite exceptional sets are never enough	36
9.1.3	Repair: make the obstruction algorithm-relative	37
9.1.4	The exact repaired universal lemma	37
9.1.5	What an algorithm-relative certificate might look like	38
9.1.6	Why certificate size, witness count, and naive geometry are not enough	38
9.1.7	A useful reinterpretation of “accessibility”	39
9.2	Reduction Robustness	39
9.3	Sensitivity and Topology: Productive Experiments, Not Automatic Lower Bounds	40
9.3.1	Boolean sensitivity	40
9.3.2	Topology of witness spaces	40
9.4	A Concrete Restricted Sandbox: What the Invariant Can Actually Prove . .	41
9.4.1	Decision trees	41
9.4.2	Sensitivity as a diagnostic rather than a universal obstruction . . .	42
9.4.3	Monotone circuits: a second sandbox	42
9.4.4	The sandbox ladder	43
9.4.5	Track C: algorithm-to-lower-bound transference	43
9.4.6	Meta-complexity as a possible bridge	44

9.5	A Proof Dependency Map	44
9.6	Could the Core Argument Still Be Saved?	45
9.6.1	Direction I: Information-theoretic obstruction	45
9.6.2	Direction II: Proof-complexity obstruction	45
9.6.3	Direction III: Circuit lower bounds	46
9.6.4	Direction IV: Algorithm-lower-bound duality	46
9.7	What the Hard-Coding Attack Has Established	46
9.7.1	An explicit rule against self-defeating invariants	46
9.8	What Would Count as a Successful New Result?	47
9.9	A Carefully Stated Main Conjecture for the Research Program	47
10	The Surviving Mathematical Core	48
10.1	The Strongest Unconditional Conclusions of the Audited Program	48
10.2	Claims Not Established	48
11	Lifting and the Semantic Extraction Target	50
11.1	Lifting Laboratory and the Concrete Extraction Target	50
11.1.1	A concrete lifting laboratory	50
11.1.2	Why the lifting laboratory still does not reach $P \neq NP$	51
11.1.3	A strengthened extraction target	52
11.1.4	Track A: Restricted computational models	52
11.1.5	Track B: Transfer toward general computation	53
11.1.6	Track C: Algorithm-to-lower-bound duality	53
11.1.7	Track D: Proof complexity	53
11.1.8	Track E: Structural invariants of witnesses	53
11.1.9	Track F: Adversarial computation	54
11.1.10	Stopping criterion	54
11.1.11	Iterative audit and diminishing returns	54
11.1.12	A possible computational experiment	56
12	Semantic Residual Extraction	57
12.1	Why a semantic extraction object is needed	57
12.2	Partial assignments and residual satisfiability	57
12.2.1	Extendibility and the terminology of partial assignments	58
12.3	Subcube interpretation and canonical extension	59
12.3.1	A monotone encoding of the information order	60
12.3.2	Extension complexity as a representation gap	61
12.4	What the prior literature changes about the research claim	61
12.5	The semantic extraction object	62
12.5.1	Witness-extension generalization	62
12.6	Time-to-circuit transfer for the extracted residual function	63

12.7	Why pointwise hard-coding is no longer enough	64
12.8	A sharp counterexample: some residual profiles are trivial	64
12.9	Representation dependence and polynomial stability	64
12.10	The residual satisfiability problem is NP-complete	65
12.11	A universal semantic family	66
12.12	The uniform circuit equivalence	66
12.13	A strong sufficient lower-bound target	67
12.13.1	The lower-bound hierarchy	67
12.14	Is the semantic object merely SAT in disguise?	67
12.15	Residual subfunction profiles and classical lower-bound connections	68
12.16	Restricted invariants: what remains useful	69
12.17	Proof complexity direction	69
12.18	Meta-complexity direction	69
12.19	Semantic extraction and lifting	69
12.20	Barrier audit for semantic extraction	70
12.20.1	Relativization	70
12.20.2	Natural proofs	70
12.20.3	Algebrization	70
12.21	Finite audit protocol for the new object	70
12.22	Classification of the new result	71
13	Structural Decomposition of the Extension Profile	72
13.1	Free-variable layers	72
13.2	Semantic mutation and contextual instability	73
13.2.1	Context profiles and the memory interpretation	74
13.2.2	Why the context route is still worth pursuing	75
13.3	The residual obstruction profile	76
13.4	Residual refutation profiles	76
13.5	Where the lower-bound target can and cannot hide	77
13.6	A structural research question	77
14	Constructive Refutation as a Lower-Bound Engine	78
14.1	Why constructiveness changes the target	78
14.2	Definition of a residual refuter	78
14.3	Three levels of residual constructivity	79
14.4	Gate elimination as a concrete laboratory	79
14.5	A contextual-interaction calibration	79
14.6	A multiplexer bridge for contextual memory	80
14.7	The residual refuter conjecture	81
14.8	Constructiveness and the original quantifier programme	81

15 Residual Meta-Complexity and Magnification	82
15.1 Exact residual circuit-size testing	82
15.2 Magnification as a possible route	82
15.3 Why meta-complexity does not trivialize the problem	83
15.4 Uniform versus nonuniform residual hardness	83
16 Limits, Status, and the Exact Frontier	84
16.1 Intermediate Concluding Position	84
A Audit Ledger	85
B Adversarial Pre-Mortem	87
C Final Mathematical Position	89
Selected Bibliography	90

Problem, Scope, and Logical Standards

1.1 The Problem and the Standard of the Attempt

The Clay Mathematics Institute currently lists the P versus NP problem among the unsolved Millennium Prize Problems. Its informal formulation is the familiar question: if a proposed solution to a computational problem can be checked quickly, must a solution itself be computable quickly? The canonical formal setting is the pair of complexity classes P and NP, with the question whether

$$P = NP \quad \text{or} \quad P \neq NP.$$

The foundational completeness theory began with the independent work of Cook and Levin in 1971; Cook's theorem placed SAT at the center of polynomial-time nondeterministic computation through polynomial-time reductions. [2, 1]

The purpose of this paper is not to repeat the standard statement and stop there. We begin from a stronger methodological question:

Can the intuitive claim that “verification is easier than generation” be formalized into an argument strong enough to force $P \neq NP$?

The answer, after careful analysis, is more subtle than either a casual “yes” or a casual “no.” There are exact theorems showing that, for canonical NP-complete problems, an efficient decision procedure can be converted into an efficient witness-finding procedure. There are also exact formulations under which an efficient certifying generator would collapse the relevant distinction. But the mere fact that current human, algorithmic, or AI methods seem to discover difficult witnesses more slowly than they verify supplied witnesses does not prove the universal lower bound required by $P \neq NP$.

That gap is the object of study here.

1.1.1 Why the gap is mathematically non-trivial

The statement $P \neq NP$ is not a claim about a particular algorithm, a particular family of algorithms, or the current state of technology. It is a universal impossibility statement. One must establish that *no* deterministic polynomial-time algorithm decides some NP language (equivalently, one NP-complete language). Consequently, the following observations, while meaningful, are logically weaker:

- (i) no presently known polynomial-time algorithm solves SAT;
- (ii) common search strategies for SAT are exponential in the worst case;
- (iii) humans and present-day AI systems often find synthesis harder than verification;
- (iv) large collections of instances exhibit search/verification asymmetry;
- (v) multiple independent auditors can often detect a flaw faster than a fresh solution can be generated.

None of (i)–(v) quantifies over *all possible polynomial-time algorithms*. The missing quantifier is the entire difficulty.

1.1.2 A methodological rule

Throughout the paper, we distinguish five levels:

1. **Formal definition:** what object is being discussed.
2. **Unconditional theorem:** what follows from the definitions and stated axioms.
3. **Conditional theorem:** what follows from an explicit additional hypothesis.
4. **Empirical or computational observation:** what has been observed on finite data or by implemented systems.
5. **Interpretive claim:** what a mathematical result may suggest philosophically or practically.

No statement is promoted from one level to another merely because it is persuasive. This distinction is especially important for P versus NP, where several intuitive arguments can be made mutually reinforcing while still leaving the central universal lower bound untouched.

Methodological audit and corrections

An earlier formulation contained the correct pressure point: the proposed “witness accessibility profile” is not yet an invariant in the mathematical sense required for an unconditional lower bound. That criticism is accepted, but it is sharpened in three ways.

First, the present programme does not assume that a witness-space metric is computationally meaningful merely because it correlates with observed difficulty. An invariant Γ must come with a theorem connecting Γ to the cost of *every* algorithm in the target class, not merely to the behavior of selected search procedures. A useful empirical correlation is therefore treated as a hypothesis-generation device rather than as lower-bound evidence.

Second, Boolean sensitivity is treated as a structural resource whose unconditional mathematics is known to be deep, but not as an already available separator of P from NP. Huang’s proof of the Sensitivity Conjecture establishes polynomial relations among classical Boolean-function measures; it does not by itself yield a lower bound against all polynomial-time algorithms. The present programme therefore uses sensitivity as an example of a successful combinatorial invariant theorem while refusing the further inference that such an invariant solves the present problem.

Third, topological data analysis of SAT witness spaces is treated as an experimental program only. Betti numbers, connectivity, or other topological descriptors may reveal useful instance geometry, but an experimentally observed phase transition would not become an asymptotic

worst-case lower bound without a theorem showing that every relevant polynomial-time algorithm must pay for that topological structure.

These corrections do not weaken the research program. They make its burden of proof explicit. The central question remains whether one can construct a *barrier-resistant*, *reduction-stable*, *algorithm-universal obstruction* to efficient solution of an NP-complete problem.

1.2 Formal Preliminaries

We work over the binary alphabet $\Sigma = \{0,1\}$ and write $|x|$ for the length of a string x . A language is a subset $L \subseteq \Sigma^*$.

1.2.1 Polynomial time

A deterministic Turing machine M runs in polynomial time if there exists a polynomial p such that, on every input x , the machine halts within $p(|x|)$ steps. The class **P** consists of languages decided by such machines.

A nondeterministic polynomial-time machine may branch into multiple computation paths; it accepts if at least one path accepts. The class **NP** consists of languages accepted by such machines.

Equivalent to the standard nondeterministic-machine formulation is the verifier formulation.

Definition 1.2.1 (Polynomially balanced verifier). *A language $L \subseteq \{0,1\}^*$ has a polynomially balanced polynomial-time verifier if there exist a polynomial q and a deterministic polynomial-time algorithm $V(x,w)$ such that*

$$x \in L \iff \exists w \in \{0,1\}^*, |w| \leq q(|x|) \text{ and } V(x,w) = 1,$$

and, for every pair (x,w) , $V(x,w)$ halts in time polynomial in $|x| + |w|$.

Theorem 1.2.2 (Verifier characterization of NP). *A language L belongs to **NP** if and only if it has a polynomially balanced polynomial-time verifier.*

Proof. The equivalence is standard. Given an NP machine, let w encode a sequence of nondeterministic choices; a deterministic verifier simulates the corresponding branch in polynomial time. Conversely, a verifier can be simulated by a nondeterministic machine that guesses a polynomial-length witness and runs the verifier. $\square$ $\square$

The key observation is immediate: an efficient verifier controls the cost of evaluating a proposed witness, not the cost of finding one.

1.2.2 Search relations

The distinction between decision and search deserves an explicit object.

Definition 1.2.3 (Witness relation). *A polynomially bounded binary relation $R(x,w)$ is a relation for which there exists a polynomial q such that whenever $R(x,w)$ holds, $|w| \leq q(|x|)$. Its associated language is*

$$L_R = \{x : \exists w R(x,w)\}.$$

If R is decidable in polynomial time, then $L_R \in \mathbf{NP}$.

Definition 1.2.4 (Total certifying generator). *A total certifying generator for R is a deterministic algorithm G such that, on input x ,*

$$G(x) = \begin{cases} w, & R(x, w) \text{ for some } w, \\ \perp, & \nexists w \text{ } R(x, w), \end{cases}$$

and G always halts.

The word “certifying” is important. A machine that outputs a witness on yes-instances but is unconstrained on no-instances is not automatically a decision procedure.

Proposition 1.2.5 (Certifying generation implies decision). *If R is polynomial-time decidable and G is a polynomial-time total certifying generator for R , then $L_R \in \mathbf{P}$.*

Proof. Compute $G(x)$. Accept exactly when $G(x) \neq \perp$. By the defining correctness condition for G , this accepts exactly the strings in L_R . Since G runs in polynomial time, so does the resulting decider. □

This elementary proposition captures one legitimate form of the intuition in the original discussion: if one can transform efficient verification into an efficient, fully certifying generation procedure, then one has collapsed the decision problem into $\mathbf{P}$.

What remains open is whether a universal version of such a transformation can be proved for NP-complete relations in the direction needed to separate $\mathbf{P}$ from $\mathbf{NP}$.

Verification, Search, and Self-Reduction

2.1 What the Verification–Generation Intuition Actually Says

The informal statement “checking is easier than finding” hides several different quantitative comparisons.

Let $R(x, w)$ be a search relation. Define the verification time

$$T_{\text{ver}}(x, w) = \text{time required to evaluate } R(x, w),$$

and define the witness-search time for a specific algorithm A by

$$T_A(x) = \text{time required for } A \text{ to return a valid } w \text{ or report failure.}$$

There is no theorem saying that

$$T_{\text{ver}}(x, w) \ll T_A(x)$$

for every algorithm A merely because R is polynomial-time decidable. In particular, A could itself be a polynomial-time algorithm if $P = NP$.

This is the first major logical boundary.

Proposition 2.1.1 (Verifier difficulty does not determine optimal generator difficulty). *The existence of a polynomial-time verifier for a relation R is compatible, as a bare statement, with both of the following possibilities:*

- (a) *there exists a polynomial-time certifying generator for R ;*
- (b) *there does not exist a polynomial-time certifying generator for R .*

Proof. Possibility (a) occurs for any problem already in P, viewed through a verifier formulation. Possibility (b) occurs whenever $L_R \notin P$ and the generator is certifying in the sense above. The point is that the verifier definition alone does not settle which case obtains. $\square$ $\square$

Therefore the verification–generation asymmetry becomes a theorem only after a lower bound is attached to generation.

2.2 Search-to-Decision for Polynomially Balanced Self-Reducible Relations

The SAT reconstruction argument is a special case of a more general principle. The distinction between decision and search is real at the level of definitions, but for many canonical NP-complete problems the distinction disappears once self-reducibility is available.

Definition 2.2.1 (Prefix-self-reducible relation). *Let $R(x, w)$ be polynomially balanced. A prefix-extension procedure for R consists of a predicate $\text{Ext}(x, u)$ on valid witness prefixes and a polynomial-time computable map f such that $\text{Ext}(x, u) \iff f(x, u) \in L_R$, where $L_R = \{x : \exists w R(x, w)\}$, and such that there is a polynomially bounded prefix encoding with*

$$\exists w R(x, w) \iff \text{Ext}(x, \varepsilon),$$

and, for every valid prefix u ,

$$\text{Ext}(x, u) \iff \text{Ext}(x, u0) \vee \text{Ext}(x, u1).$$

The length of a witness prefix is bounded by a polynomial in $|x|$.

Theorem 2.2.2 (Prefix reconstruction). *Let R be a polynomial-time decidable, polynomially balanced relation admitting a prefix-extension procedure in the sense above. If $L_R \in \mathbf{P}$, then there is a polynomial-time algorithm that, on every $x \in L_R$, outputs a witness w with $R(x, w) = 1$, and on every $x \notin L_R$ outputs $\perp$.*

Proof. Because $L_R \in \mathbf{P}$ and Ext is polynomial-time reducible to L_R , the extension predicate can be decided in polynomial time. Starting with $u = \varepsilon$, query whether $\text{Ext}(x, u0)$ holds. If so, replace u by $u0$; otherwise replace u by $u1$. The defining identity guarantees that, whenever an extension exists, at least one of the two branches remains extendible. After at most polynomially many steps the resulting prefix is a complete witness. The algorithm first queries $\text{Ext}(x, \varepsilon)$, so it returns $\perp$ exactly on no-instances. Each query has polynomial length and there are only polynomially many queries. Hence the total running time is polynomial. $\square$

Corollary 2.2.3 (SAT search–decision equivalence). *For SAT, a polynomial-time decision algorithm implies a polynomial-time witness-producing algorithm, and a polynomial-time total witness-producing algorithm implies a polynomial-time decision algorithm. Thus the decision and constructive-search formulations are polynomial-time equivalent.*

Remark 2.2.4. *This theorem closes one loophole in the informal verification–generation narrative. For SAT, it is not legitimate to rescue a hypothetical $P = NP$ world by claiming that the decision problem might be easy while witness construction remains intrinsically difficult. The hard question is therefore not whether decision and search are formally different tasks; it is whether a polynomial-time algorithm exists for the NP-complete decision/search problem at all.*

2.3 The Exact Search-to-Decision Equivalence for SAT

The most useful rigorous response to the intuition is not to deny it but to sharpen it.

For Boolean formulas, decision and witness construction are polynomially interreducible. In particular, if SAT is in $\mathbf{P}$, then satisfying assignments can be found in polynomial time. This means that for SAT one cannot evade the $\mathbf{P}$ versus $\mathbf{NP}$ question by saying that perhaps deciding existence is easy but constructing an actual witness is hard. Under the standard encoding, efficient decision suffices to recover a witness.

2.3.1 Restricted prefix formulas

Let $\varphi(z_1, \dots, z_n)$ be a Boolean formula. For a partial assignment $a = (a_1, \dots, a_k) \in \{0, 1\}^k$, let φ_a denote the formula obtained by substituting $z_i = a_i$ for $1 \leq i \leq k$.

Lemma 2.3.1 (Prefix extension). *If φ is satisfiable and a is a satisfiable partial assignment, then at least one of the two extensions $a0$ or $a1$ is satisfiable.*

Proof. Let b be a complete satisfying assignment extending a . Then b_{k+1} is either 0 or 1. If $b_{k+1} = 0$, $a0$ extends to b and is satisfiable; if $b_{k+1} = 1$, $a1$ is satisfiable. $\square$ $\square$

Theorem 2.3.2 (SAT search-to-decision reconstruction). *Suppose there is a deterministic polynomial-time decider D for SAT. Then there is a deterministic polynomial-time algorithm S which, on every satisfiable formula φ with n variables, outputs a satisfying assignment, and on every unsatisfiable formula outputs “UNSAT”.*

Proof. Run $D(\varphi)$. If $D(\varphi) = 0$, output “UNSAT.” If $D(\varphi) = 1$, construct a satisfying assignment bit by bit.

Start with the empty partial assignment $a^{(0)}$. At stage k , test the two restricted formulas $\varphi_{a^{(k-1)}0}$ and $\varphi_{a^{(k-1)}1}$. By the prefix-extension lemma, at least one is satisfiable. Choose, for example, the branch $b \in \{0, 1\}$ for which $D(\varphi_{a^{(k-1)}b}) = 1$, and set

$$a^{(k)} = a^{(k-1)}b.$$

After n stages, $a^{(n)}$ is a satisfying assignment.

There are $n + 1$ calls to D up to a constant factor, each on an input of size polynomially related to $|\varphi|$. Hence the total running time is polynomial in $|\varphi|$. $\square$ $\square$

Corollary 2.3.3. *If $\text{SAT} \in P$, then $P = NP$.*

Proof. SAT is NP-complete under polynomial-time many-one reductions, so $\text{SAT} \in P$ implies $NP \subseteq P$. The reverse inclusion is standard. $\square$ $\square$

This theorem is structurally important. It says that, for SAT, decision and constructive search are not separate complexity phenomena at the polynomial level. Therefore any successful proof of $P \neq NP$ can be framed either as a decision lower bound or as a search lower bound, but the lower bound must quantify over all polynomial-time algorithms.

Auditing, Finite Evidence, and False Shortcuts

3.1 A Precise Formalization of the “Audit Team” Intuition

A recurring idea in practical reasoning is that several independent auditors may be substantially better at falsifying a proposed construction than a single creator is at producing a correct construction.

This is a reasonable engineering principle. To use it mathematically, we need a model.

Definition 3.1.1 (Audit system). *For a family of instances x and candidate objects y , an audit system of width m is a tuple*

$$\mathcal{A} = (V_1, \dots, V_m),$$

where each V_i is an algorithm that returns either “accept,” “reject,” or “inconclusive.” The combined audit rule is a deterministic function of the m outputs.

The central limitation is immediate: if each V_i is polynomial-time, then the combined audit is polynomial-time for fixed m . Such an audit can certify that a candidate fails a specified collection of tests, but it does not search the entire space of possible candidates.

Proposition 3.1.2 (Fixed-width auditing does not create a P-vs-NP separation). *Let m be fixed. If each V_i runs in polynomial time and the combined rule is polynomial-time, then the resulting audit system is itself polynomial-time. Its existence alone therefore does not imply $P \neq NP$.*

Proof. Run $V_1, \dots, V_m$ and then evaluate the combined rule. A constant number of polynomial-time procedures composed with polynomial-time postprocessing remains polynomial-time. $\square$

$\square$

One might object that a large number of auditors can run in parallel. That observation is valid but does not change the logical issue. Parallelism can reduce wall-clock time under a parallel cost model, but P versus NP is traditionally phrased in a sequential polynomial-time model. More importantly, even unlimited parallel checking of a *given* candidate does not produce a candidate unless some search mechanism explores the candidate space.

3.1.1 Error detection versus universal certification

There are two very different tasks:

Local falsification: $\exists i V_i(x, y) = \text{reject}$,
 Universal non-existence: $\nexists y V(x, y) = 1$.

The first can be witnessed by a single counterexample. The second requires a global argument that no valid witness exists.

This asymmetry is real and important. But SAT and NP-completeness are precisely organized around the difficulty of proving that the global existential condition is true or false without enumerating exponentially many candidates.

Remark 3.1.3. *The statement “it is easy to refute a bad candidate” is weaker than “it is easy to determine whether some good candidate exists.” The latter is the NP decision problem.*

3.2 Finite Ensembles of Auditors

The motivating conversation places considerable weight on heterogeneous auditors: several independent systems may be much better at discovering flaws than a single system. This can be formalized, but the formal consequence is weaker than the intuition suggests.

Theorem 3.2.1 (Finite audit closure). *Let $V_1, \dots, V_k$ be deterministic polynomial-time procedures and let k be fixed independently of the input length. Any fixed finite Boolean combination of their outputs is computable in polynomial time.*

Proof. Run the k procedures sequentially or in parallel and evaluate the fixed Boolean combination. If V_i runs in time n^{c_i} , the total sequential time is $O(kn^c)$ where $c = \max_i c_i$, hence polynomial because k is constant. $\square$

Proposition 3.2.2 (Finite diversity is not a universal lower bound). *Agreement, disagreement, or complementarity among a fixed finite ensemble of polynomial-time auditors cannot by itself establish that an NP-complete language lies outside P.*

Proof. The ensemble itself remains a polynomial-time computation. Its collective behavior therefore supplies, at most, a particular polynomial-time predicate about the input and the proposed evidence. A statement that the ensemble fails on some inputs would establish a lower bound against that ensemble; it would not establish a lower bound against every polynomial-time machine. The latter contains the former as a special case but is quantified over an unbounded collection of algorithms and polynomial running-time bounds. $\square$

Remark 3.2.3. *This does not make heterogeneous auditing mathematically irrelevant. It makes its correct role precise. Independent auditors are excellent tools for falsification, error localization, proof checking, and model comparison. They become a complexity-theoretic lower-bound method only if their collective behavior can be embedded in a theorem that quantifies over the full target class.*

3.3 Finite Evidence, Adversarial Search, and the Limits of Testing

The argument that repeated testing should eventually license a binary decision is often excellent scientific practice for engineering systems. It does not establish a worst-case theorem over an infinite input domain.

Proposition 3.3.1 (Finite testing cannot settle an unrestricted language identity). *Let $S \subseteq \{0,1\}^*$ be a finite test set. For any language $L \subseteq \{0,1\}^*$, there exists a distinct language $L' \subseteq \{0,1\}^*$ such that*

$$L' \cap S = L \cap S.$$

Proof. Choose any string $z \notin S$. Define $L' = L \Delta \{z\}$, where Δ denotes symmetric difference. Then L' agrees with L on S but differs on z . $\square$ $\square$

This elementary result does not imply that experiments are useless. Rather, it tells us exactly what experiments can and cannot establish without additional structural assumptions. Experiments can identify bugs, falsify candidate mechanisms, discover invariants, and support conjectures. They cannot by themselves prove an unrestricted universal complexity lower bound.

3.3.1 A stronger adversarial observation

Suppose a proposed algorithm A has been tested on every instance of size at most N . This is still finite evidence. A language or problem can agree with the observed behavior through size N and diverge later. Complexity lower bounds are asymptotic and worst-case by nature.

Therefore the correct role of experimentation in this project is diagnostic:

$$\text{experiment} \rightarrow \text{conjecture} \rightarrow \text{structural lemma} \rightarrow \text{proof},$$

not

$$\text{experiment} \rightarrow \text{proof}.$$

3.4 Gödel Incompleteness: Relevant Warning, Invalid Shortcut

A natural line of thought is that a formal system cannot completely verify itself, and therefore a sufficiently sophisticated self-referential computational theory should somehow force $P \neq NP$. The analogy is intellectually interesting, but the direct implication is invalid.

Gödel's incompleteness theorems concern formal theories that are sufficiently expressive to represent arithmetic and that satisfy appropriate consistency conditions. Roughly, they imply that certain such theories contain sentences that are true in the intended model but not provable inside the theory, or that the theory cannot internally prove its own consistency under the usual hypotheses.

The P versus NP question is a complexity-class statement about computational resources. It asks whether every polynomially verifiable language is polynomial-time decidable. There is no logical step of the form

$$\text{incompleteness} \implies P \neq NP$$

known in standard complexity theory.

Indeed, an NP verifier need not be a self-verifying formal theory in the Gödelian sense. It is simply an efficient relation-checking algorithm. Verification of a purported certificate is not the same as proving the consistency of the entire formal system containing the verifier.

Proposition 3.4.1 (No direct Gödel implication). *The existence of a formal system exhibiting Gödel incompleteness, by itself, does not establish $P \neq NP$.*

Proof. The proposition is a statement about logical implication. Gödel incompleteness applies to a specified class of formal theories. P versus NP concerns complexity classes defined over

algorithms. A direct implication would require an additional theorem connecting incompleteness to a superpolynomial deterministic time lower bound for an NP-complete language. No such implication follows merely from the hypotheses of incompleteness. In particular, the incompleteness theorem does not assert that every efficiently verifiable existential condition lacks an efficient deterministic decision procedure. $\square$ $\square$

There are, however, deep connections between bounded arithmetic, proof complexity, feasible reasoning, and complexity classes. Those connections are not irrelevant. They may constrain what can be proved in restricted theories and they provide alternative formulations of lower-bound questions. But they are a research program, not a one-line reduction from incompleteness to $P \neq NP$.

3.5 The Epistemic Regress and the Meaning of “Proof”

The worry that proof verification leads to an infinite regress can be formulated precisely:

Why trust proof π ? Because verifier V accepts it.

Then:

Why trust V ? Because a second verifier accepts V .

And so on.

This is a legitimate epistemological question. It is not, however, a defect in the mathematical definition of a proof.

A formal proof is a finite syntactic object checked according to a specified inference relation. The statement proved is conditional on the chosen axioms and rules. One may philosophically ask whether the axioms or the formal system are “really” sound, but that is a different question from whether the proof is valid *in the formal system*.

This distinction lets us avoid two opposite mistakes:

- (a) demanding an infinite regress of proofs of proofs of proofs before accepting any finite theorem;
- (b) treating human acceptance or repeated experimental confirmation as equivalent to formal derivation.

The appropriate stopping rule is finite and explicit: fix the formal system, state assumptions, verify the derivation, and separately record any unresolved foundational or philosophical question.

For the present project this means that “rigor for its own sake” is not the standard. The standard is *finite auditability of the mathematical dependency graph*. Once the theorem follows from stated assumptions, a further demand that every axiom itself be proven inside an even stronger system changes the question rather than strengthening the original theorem.

The Quantifier Gap

4.1 The Quantifier Ledger

A recurring source of error is the silent movement between four different quantifier patterns. It is useful to display them explicitly.

For a fixed algorithm A ,

$$\exists n \exists x \in \{0, 1\}^n \quad A(x) \text{ fails.}$$

For a finite collection of algorithms,

$$\forall i \leq k \exists n_i \exists x_i \quad A_i(x_i) \text{ fails.}$$

For all polynomial-time algorithms with a fixed exponent bound c ,

$$\forall A \in \mathcal{M}_c \exists^\infty n \exists x \in \{0, 1\}^n \quad A(x) \text{ fails.}$$

The unrestricted separation $L \notin \mathbf{P}$ requires the quantifier to range over every polynomial-time machine and every polynomial exponent:

$$\forall A \in \mathbf{P}\text{-machines} \exists x \quad A(x) \text{ fails on } L.$$

The distinction is not cosmetic. A proof that defeats one algorithm, a finite family, or even every algorithm in one restricted architecture class does not automatically quantify over all polynomial-time computation.

Proposition 4.1.1 (Quantifier-gap criterion). *Any proof of $L \notin \mathbf{P}$ must contain an argument whose effective scope covers every deterministic polynomial-time decider for L , not merely an empirically large or adversarially selected family of algorithms.*

Proof. By definition, $L \notin \mathbf{P}$ means that no deterministic polynomial-time machine decides L . Negating the definition gives the universal quantification over all such machines. Any purported proof whose final adversary does not cover that collection establishes at most a restricted lower bound. $\square$

Remark 4.1.2. *This is the exact mathematical version of the “quantifier gap” identified in the earlier formulation. It is stronger than the sociological observation that no known algorithm has succeeded: the target is a theorem with the correct order of quantifiers.*

4.2 What Would a Direct Proof of $P \neq NP$ Have to Establish?

The cleanest target is the following.

Theorem 4.2.1 (Target form). *To prove $P \neq NP$, it suffices to exhibit a language $L \in NP$ such that $L \notin P$.*

Because SAT is NP-complete, it suffices to prove

$$\text{SAT} \notin P.$$

Equivalently, one must show that for every deterministic polynomial-time machine M , there exists an input on which M gives an incorrect answer for SAT. The alternative time-bound formulation is obtained by quantifying over arbitrary deterministic machines rather than machines already assumed to run in polynomial time: every deterministic machine either errs on some input or has superpolynomial worst-case running time.

The universal quantifier can be written schematically as

$$\forall M \in \text{DTIME}(n^{O(1)}), \quad M \text{ does not decide SAT.}$$

This is the central obstacle. A proof cannot merely defeat a known class of search heuristics. It must defeat every possible polynomial-time SAT algorithm, including algorithms not yet conceived and algorithms that do not resemble any currently studied method.

4.2.1 A proof-by-diagonalization template

A natural idea is to enumerate deterministic polynomial-time machines

$$M_1, M_2, M_3, \dots$$

and construct a language L that disagrees with each M_i on some input. This is the broad diagonalization instinct.

For unrestricted computability, diagonalization is powerful. For P versus NP, however, the desired language must simultaneously be in NP and outside P. Constructing such a language while preserving the efficient-verification requirement is the hard part.

Moreover, diagonalization arguments of the basic oracle-respecting kind are subject to the relativization barrier discussed below.

From Intuition to a Lower-Bound Program

5.1 The First Strong Candidate: A Verification-to-Generation Gap Axiom

The intuition from the motivating discussion can be stated as a precise hypothesis.

Conjecture 5.1.1 (Universal verification-to-generation gap). *There exists an NP-complete language L with a polynomial-time verifier V such that no deterministic polynomial-time algorithm can, on every $x \in L$, produce a valid witness for V within polynomial time while also certifying non-membership by a polynomial-time failure output.*

If this conjecture were proved, then $P \neq NP$ would follow immediately by the certifying-generator proposition. But the conjecture is essentially a restatement of the core hardness issue in search language. It does not solve the problem by itself.

This is still useful because it tells us exactly what a stronger theorem must provide: not a statement about existing solvers, but a lower bound that rules out every polynomial-time certifying search mechanism.

5.1.1 A more structured target

A more ambitious formulation is a resource-separation principle.

Definition 5.1.2 (Uniform verification-search gap). *A family of NP relations R_n has a uniform verification-search gap $g(n)$ if:*

- (a) *verification of $R_n(x, w)$ takes time polynomial in n and $|w|$;*
- (b) *every certifying generator for R_n requires worst-case time at least $g(n)$ on infinitely many inputs.*

A proof of a superpolynomial $g(n)$ for an NP-complete relation would imply $P \neq NP$. An exponential lower bound would be stronger. A merely superpolynomial lower bound already suffices.

The difficulty is that known lower bounds usually apply to restricted computational models, proof systems, circuit classes, communication protocols, or other structured resources, rather than to all deterministic polynomial-time algorithms.

5.2 Barrier I: Relativization

The first major barrier to naive diagonalization was identified by Baker, Gill, and Solovay. They constructed recursive oracles A and B such that

$$P^A = NP^A \quad \text{and} \quad P^B \neq NP^B.$$

Thus a proof technique that relativizes in a sufficiently black-box fashion cannot settle the unrelativized question. [3]

Remark 5.2.1 (Interpretation). *The result does not say that P and NP are independent in the ordinary sense, nor that no diagonal argument can ever succeed. It says that a broad class of arguments remains valid in oracle worlds where the answer changes. Hence such an argument cannot by itself distinguish the real world from both oracle possibilities.*

This matters directly for the audit-team intuition. If the proof only treats algorithms as black boxes through their input-output behavior and the proof continues to work in arbitrary oracle extensions, then the oracle theorem warns that the method is too coarse.

A successful proof must exploit some structural property of ordinary computation that is destroyed or altered by arbitrary relativization.

5.3 Barrier II: Natural Proofs

Razborov and Rudich introduced the notion of natural proofs and showed, under standard hardness assumptions, that broad classes of combinatorial circuit lower-bound arguments cannot establish superpolynomial general circuit lower bounds. [4]

The relevance is indirect but important. Many plausible strategies for $P \neq NP$ attempt to identify an easily testable structural property shared by “simple” computations and absent from a hard function. If that property is sufficiently constructive and large, the natural-proofs framework warns that it may be incompatible with standard cryptographic hardness assumptions.

The lesson is not that combinatorics is useless. Rather, a successful global lower bound must be sufficiently delicate to avoid being both broad and efficiently recognizable in the forbidden sense.

5.4 Barrier III: Algebrization

Aaronson and Wigderson showed that relativization and natural proofs do not exhaust the known obstacles: a broader class of algebraic-relativizing arguments also fails to resolve P versus NP . They introduced “algebrization” as a barrier capturing many techniques that extend oracle access by low-degree algebraic extensions. [5]

Therefore, an eventual proof must do more than merely outwit the original diagonalization and avoid the standard natural-proof template. It must engage with a form of structure that survives neither ordinary oracle simulation nor the usual algebraic extension framework.

This is one reason that a superficially compelling universal lower bound should be treated with suspicion: there are known families of methods that can produce impressive lower bounds while systematically stopping short of the full target.

5.5 What the Barriers Tell Us About the Proposed Program

The verification-versus-generation approach can be evaluated against the three barriers.

1. If the approach merely says “every verifier is easy to simulate and every generator must search many candidates,” it is too informal to quantify over all algorithms.
2. If it reduces hardness to a broad efficiently testable property of Boolean functions, it risks falling within the natural-proofs framework.
3. If its core transformation relativizes or algebrizes unchanged, the known oracle constructions warn that it cannot settle the unrelativized problem.

Thus any genuine continuation of this project should explicitly state what structural feature of computation is being exploited and why the resulting argument escapes these barriers.

Resource Transfer, Proof Complexity, and Structural Routes

6.1 A More Productive Route: Search, Proof Systems, and Witness Structure

The original intuition is perhaps more naturally connected not to “P versus NP” in one jump, but to the theory of proof complexity and NP witness systems.

A propositional proof system is, informally, a polynomial-time verifier that accepts encodings of valid proofs. Cook and Reckhow’s framework makes the connection precise: proving strong lower bounds on propositional proof systems is related to separating NP from coNP, and hence would imply a P-versus-NP separation. Modern surveys of proof complexity emphasize the equivalence between the existence of polynomially bounded proof systems for UNSAT and $NP = coNP$. [10]

Theorem 6.1.1 (Proof-system implication). *If one proves that no polynomially bounded propositional proof system exists for UNSAT, then*

$$NP \neq coNP,$$

and therefore

$$P \neq NP.$$

Proof. If a polynomially bounded proof system for UNSAT existed, then UNSAT would lie in NP. Since UNSAT is coNP-complete, this gives $coNP \subseteq NP$. Taking complements yields $NP \subseteq coNP$, hence $NP = coNP$. Therefore, proving that no polynomially bounded proof system exists implies $NP \neq coNP$. Since $P = NP$ would imply $NP = coNP$, the contrapositive gives $P \neq NP$. □

This is a powerful reformulation because it transforms the vague statement “discovery is harder than verification” into a lower bound on the size of certificates in a very explicit verification architecture.

The difficulty remains universal: one must lower-bound proof length for *every* sound and complete proof system of the relevant kind, not merely for one chosen calculus such as Resolution.

6.1.1 Why restricted proof-system lower bounds still matter

Even when they do not solve P versus NP, strong lower bounds for Resolution, cutting planes, bounded-depth Frege, polynomial calculus, sum-of-squares, and related systems are genuine

mathematical progress. They identify computational resources that provably fail on explicit families.

That suggests a refined research strategy:

$$\begin{aligned} &\text{verification intuition} \rightarrow \text{certificate structure} \rightarrow \text{resource-sensitive lower bound} \\ &\quad \rightarrow \text{barrier audit} \rightarrow \text{possible universal theorem.} \end{aligned}$$

This is more promising than arguing directly from the psychology of search.

6.2 A Formal “Creator–Auditor” Model

To make the original intuition as strong as possible, we define a creator and a family of auditors.

Definition 6.2.1 (Creator). *A creator is an algorithm C that receives an instance x and either outputs a candidate witness w or reports failure.*

Definition 6.2.2 (Auditor family). *An auditor family is a sequence $(A_i)_{i=1}^m$ of algorithms such that each $A_i(x, w)$ is polynomial-time and the combined system is sound: if some auditor rejects, then w is not an acceptable witness.*

The creator-auditor asymmetry can be measured by a ratio such as

$$\rho_C(x) = \frac{T_C(x)}{T_{\text{audit}}(x, w)},$$

where defined. Large values of ρ_C are possible in practice.

But a complexity-theoretic lower bound would require something like:

$$\inf_C \sup_{x \in L, |x|=n} T_C(x) \geq g(n),$$

where g is superpolynomial.

This is a fundamentally different statement. The infimum ranges over *all* creators. Existing empirical observations usually concern a finite or tiny subset of this algorithmic space.

Lemma 6.2.3 (Algorithmic-space quantifier gap). *Suppose a family of experiments rules out a finite collection $C_1, \dots, C_m$ of candidate generators. This does not imply that all polynomial-time generators have been ruled out unless an additional structural theorem proves that every polynomial-time generator belongs to the tested family or is reducible to it in a manner preserving the lower bound.*

Proof. The statement is immediate from quantifier structure. Ruling out finitely many algorithms establishes

$$\neg C_1 \wedge \dots \wedge \neg C_m,$$

not

$$\forall C \in \mathcal{P}, \neg C,$$

where $\mathcal{P}$ denotes the class of all polynomial-time algorithms. The latter requires an additional completeness argument. □ □

This lemma looks elementary, but it is precisely the point at which many persuasive informal arguments silently overreach.

6.3 A Conditional Theorem That Captures the Intended Logic

The previous conditional theorem was stated for an arbitrary NP verifier, but the search-to-decision step silently requires a suitable self-reducibility property. We now state the result in a form that makes the dependency explicit and uses SAT, where the reconstruction is elementary.

Theorem 6.3.1 (Conditional SAT search lower bound implies $P \neq NP$). *Assume that there is a function $g : \mathbb{N} \rightarrow \mathbb{N}$ growing faster than every polynomial such that every deterministic algorithm that correctly solves the SAT search problem has, for infinitely many input lengths n , some satisfiable formula φ of length n on which it requires more than $g(n)$ steps.*

Then

$$P \neq NP.$$

Proof. Suppose, for contradiction, that $P = NP$. Since SAT is NP-complete, SAT is in P . Hence there is a polynomial-time decision procedure D for SAT.

For a formula φ with variables

$$z_1, \dots, z_m,$$

construct a satisfying assignment by repeated restriction. At stage i , query D on

$$\varphi[z_1 = 0]$$

after the previously selected values have been substituted. If the restricted formula is satisfiable, set $z_i = 0$; otherwise set $z_i = 1$. At each stage the chosen partial assignment remains extendible to a satisfying assignment. After at most m iterations, a complete satisfying assignment is obtained.

Because each restricted formula has size polynomial in $|\varphi|$ and there are only polynomially many calls to D , the resulting search algorithm runs in polynomial time. This contradicts the assumed superpolynomial lower bound on every correct SAT search algorithm.

Therefore SAT is not in P , hence $P \neq NP$. □ □

Remark 6.3.2. *The theorem is elementary but strategically important. It tells us exactly what the offensive program must accomplish: not a lower bound for a particular solver, not a lower bound for a particular family of heuristics, and not a lower bound for raw enumeration, but a worst-case lower bound against every polynomial-time SAT search algorithm.*

6.4 Attempted Proof of the Universal Generator Lower Bound

We now attack the repaired problem directly. The objective is to construct an algorithm-relative obstruction that survives the hard-coding counterargument and still defeats every polynomial-time algorithm.

6.4.1 Step 1: enumerate polynomial-time algorithms

Fix an effective enumeration

$$A_1, A_2, \dots$$

of deterministic Turing-machine descriptions together with explicit polynomial clock bounds. Every polynomial-time algorithm occurs somewhere in this enumeration.

A classical diagonal strategy would attempt, for each A_i , to construct an input x_i on which A_i fails.

6.4.2 Step 2: the standard diagonal instinct

For deterministic time hierarchy separations, one simulates the machine being diagonalized and deliberately disagrees with its answer. The attraction of the method is that it directly attacks an enumeration of algorithms.

However, P versus NP asks for a language in NP that defeats every polynomial-time deterministic machine. The NP side imposes a severe constraint: membership must possess a polynomially bounded, polynomial-time checkable witness.

Thus the naive diagonal language must simultaneously satisfy

$$L \in NP$$

and

$$\forall A \in P \quad A \text{ fails to decide } L.$$

The usual deterministic time hierarchy construction does not automatically provide this combination.

6.4.3 Step 3: try to encode the diagonal computation into the witness

A natural attempt is to let a witness contain a computation tableau certifying that the allegedly fast machine A_i behaves incorrectly on a specially constructed input.

Local consistency of a bounded computation tableau is polynomial-time checkable. This is exactly the idea underlying Cook's use of tableaux in the NP-completeness theory. The difficulty is not checking a finite tableau. The difficulty is arranging the tableau so that the resulting NP language provably defeats every polynomial-time machine without allowing the language definition itself to perform an operation outside polynomial-time verifiability.

For a deterministic machine whose exact behavior is being simulated, one can in principle describe a finite computation history. But to make the diagonal language disagree with *every* polynomial-time machine while remaining in NP, the construction must control the quantifier

$$\forall i$$

without requiring an NP verifier to certify a universal negative about arbitrary machines.

This is the precise obstruction, not merely “diagonalization feels hard.”

6.4.4 Step 4: why the obvious self-reference does not finish the proof

One may try to define a language whose yes-witness contains a bounded computation showing that the machine indexed by the input fails. There are two cases.

If the verifier asks only for a locally checkable certificate of failure, the construction can remain in NP. But then one must prove that for every polynomial-time decider there is an input for which such a certified failure occurs, and that proof is itself the lower-bound argument.

If instead the verifier directly checks a claim such as

$$A_i(x) \neq L(x)$$

without a bounded certificate of the relevant computation, the verifier risks leaving NP because it is being asked to establish a universal semantic property rather than a single polynomially bounded computation.

Thus the self-reference does not magically manufacture the missing lower bound. It merely relocates the problem to the design of the diagonal language.

6.4.5 Step 5: the relativization test

There is a second, independent attack. Baker, Gill, and Solovay constructed recursive oracles A and B satisfying

$$P^A = NP^A \quad \text{and} \quad P^B \neq NP^B.$$

Therefore any proposed proof whose essential steps continue to hold unchanged in the presence of arbitrary oracles cannot settle the unrelativized question.

Our diagonal construction as currently formulated uses only machine enumeration, bounded simulation, and ordinary black-box computation. Those operations have the same form in the oracle model. Consequently the construction has not yet supplied the non-relativizing ingredient required to distinguish the two oracle worlds. [3]

6.4.6 Step 6: a repaired diagonal target

The correct next target is therefore not “find a diagonal language” in isolation. It is to construct a diagonal argument with an additional operation Q satisfying all of the following:

- (i) Q preserves NP membership;
- (ii) Q produces a worst-case input against every polynomial-time algorithm;
- (iii) Q cannot be relativized as a black-box oracle construction;
- (iv) Q is stable under polynomial reductions;
- (v) Q yields an explicit, checkable lower-bound certificate.

The fourth and fifth conditions prevent the argument from becoming merely a bespoke language construction whose hardness cannot be transferred to SAT.

6.4.7 The revised missing lemma

We can now state the actual unresolved bridge.

Conjecture 6.4.1 (Barrier-resistant adversarial certificate lemma). *There exists an explicit NP-complete search relation R and a polynomial-time checkable certificate system producing values*

$$\Gamma(A, x)$$

such that:

- (a) Soundness. *Whenever A correctly solves R ,*

$$T_A(x) \geq \Gamma(A, x).$$

- (b) Uniform adversariality. *For every deterministic polynomial-time A , there are arbitrarily large n and inputs x of length n with*

$$\Gamma(A, x) > n^c$$

for every fixed c .

- (c) Reduction stability. *Polynomial-time changes of representation can change Γ by at most polynomial factors, with explicitly bounded distortion.*

- (d) Non-relativization. *The proof of (b) uses information not preserved by arbitrary oracle extensions.*
- (e) Barrier resistance. *The argument does not reduce to a natural proof of general circuit lower bounds or to an algebrizing argument of the known blocked type.*

Remark 6.4.2. *Unlike the earlier pointwise $\Gamma(x)$ proposal, this conjecture survives the hard-coding theorem. The algorithm is fixed first, and the hard instance is chosen afterwards. The challenge is correspondingly much harder and much more faithful to the true quantifier structure of P versus NP .*

6.5 From Algorithm-Relative Certificates to Resource Transfer

A central audit observation deserves a further refinement. Changing an invariant from $\Gamma(x)$ to $\Gamma(A, x)$ repairs the hard-coding quantifier problem, but it does not by itself create a non-circular lower-bound mechanism. An algorithm-relative quantity can be defined tautologically, for example by taking $\Gamma(A, x) = T_A(x)$, or by taking a quantity that is merely an encoding of the computation of A on x . Such a definition is formally sound but mathematically unhelpful: it has renamed the bottleneck rather than explaining it.

The programme therefore replaces the vague requirement of an “algorithm-relative invariant” by a more concrete requirement: a *resource-transfer certificate*. The certificate must connect an arbitrary algorithm to a resource for which an independent lower bound can be proved. The architecture is

$$\begin{aligned} \text{algorithm } A &\longrightarrow \text{extracted resource object } E(A) \\ &\longrightarrow \text{resource lower bound} \longrightarrow \text{time lower bound for } A. \end{aligned}$$

6.5.1 Resource-transfer certificates

Definition 6.5.1 (Resource-transfer certificate). *Let R be a polynomially verifiable search relation. A resource-transfer certificate scheme consists of a resource class $\mathcal{M}$, an extraction map E assigning an object $E(A) \in \mathcal{M}$ to each admissible algorithm A , a resource functional $r(M, x) \in \mathbb{N}$, a polynomial transfer bound*

$$r(E(A), x) \leq p(T_A(x), |x|),$$

whenever A succeeds on x , and a separate lower-bound theorem forcing $r(E(A), x)$ to be large on an adversarial family.

Theorem 6.5.2 (Resource-transfer principle). *Suppose R is an NP-complete self-reducible search relation and a resource-transfer certificate scheme exists. If there is a superpolynomial function h such that every deterministic polynomial-time correct solver A for R has arbitrarily large inputs x satisfying*

$$r(E(A), x) \geq h(|x|),$$

then $P \neq NP$.

Proof. Assume $P = NP$. By self-reducibility and NP-completeness, R has a deterministic polynomial-time search algorithm A with running time bounded by some polynomial q . The transfer inequality gives

$$r(E(A), x) \leq p(q(|x|), |x|),$$

which is polynomial in $|x|$. The assumed adversarial lower bound gives infinitely many x with $r(E(A), x) \geq h(|x|)$, where h dominates every polynomial. For sufficiently large $|x|$ the two inequalities are incompatible. Hence $P \neq NP$. $\square$ $\square$

This theorem is deliberately conditional. Its value is that it separates the unresolved problem into two independently attackable obligations: (1) extract a meaningful resource from an arbitrary fast algorithm, and (2) prove a superpolynomial lower bound on that resource.

6.5.2 Why algorithm-relative is necessary but not sufficient

The hard-coding theorem forces the order $A \rightarrow x$, but that is only the first constraint. A useful $\Gamma(A, x)$ must also satisfy

$$\Gamma(A, x) \leq T_A(x)$$

by an independently provable argument, without defining Γ from T_A itself. Thus

$$\boxed{\text{algorithm-relative} + \text{independently analyzable} + \text{transferable}}$$

is the actual requirement.

Proposition 6.5.3 (Syntactic padding obstruction). *A purely syntactic statistic of an algorithm description cannot by itself be a robust lower-bound certificate. Given a correct algorithm A , one may construct another correct algorithm A' computing the same function but containing an arbitrary amount of unreachable description padding.*

Proof. Insert unreachable states or instructions before the code that executes A . The computed function and running time on all executions remain unchanged, while any statistic counting inert syntactic material can be made arbitrarily large. $\square$ $\square$

Therefore a proposed certificate must either be invariant under an appropriate semantic equivalence or have a theorem showing that the chosen syntactic normal form is canonical for the relevant lower-bound argument.

Adversarial Attacks on Candidate Resources

7.1 Kolmogorov and Circuit Complexity: Two Reviewer Suggestions Under Direct Attack

Proposition 7.1.1 (Plain conditional Kolmogorov complexity cannot lower-bound the running time of a fixed total generator). *Let G be a fixed total computable witness generator. Then there is a constant c_G such that*

$$K(G(x) \mid x) \leq c_G$$

for every input on which G returns a witness, up to the fixed universal-machine convention.

Proof. A constant-length program can contain a description of G , read x , simulate $G(x)$ until it halts, and print its output. The program length depends on G but not on x . $\square$ $\square$

Every polynomially balanced decidable NP relation admits a total computable generator by exhaustive search. Consequently plain conditional Kolmogorov complexity cannot be the desired unconditional time obstruction. Time-bounded Kolmogorov complexity is more relevant, but then the time parameter has entered the definition and the missing lower-bound problem has merely moved into a meta-complexity setting.

The broader literature makes this direction mathematically serious: meta-complexity studies problems in which computational resources are themselves part of the input or task. Such problems can be hard even though they concern properties of computations rather than ordinary instances of a single language. They do not, by themselves, yield a universal SAT time lower bound.

Proposition 7.1.2 (Polynomial-length witnesses cannot have superpolynomial ordinary circuit complexity). *Every binary string of length m has a Boolean circuit of size $O(m)$ that produces it by hard-wiring its bits.*

Proof. Use one constant-size output gadget per bit and wire the gadgets directly to the outputs. $\square$ $\square$

Hence the suggestion that an NP verifier could force every valid witness, or a single explicit polynomial-length configuration, to have superpolynomial ordinary circuit complexity cannot work in the standard explicit-input setting. A time-bounded, succinct, distributional, or family-level complexity measure remains possible, but the naive witness-level formulation is eliminated by the proposition.

7.2 Direct Attack on the Dynamic Communication-Trace Proposal

An earlier audit proposed a concrete instantiation of the extraction map based on the dynamic communication trace of a Turing-machine computation. The proposal is valuable because it replaces an abstract “accessibility” score by an explicit object: a computation tableau together with a partition of its information flow. We therefore test it directly rather than accepting it as a promising slogan.

The proposal has two logically distinct components:

- (i) an *extraction step*, converting a computation of an arbitrary solver into a communication object; and
- (ii) a *lower-bound step*, proving that every solver must incur a large amount of communication across the chosen partition.

The first component is natural. The second is where the decisive difficulty appears.

7.2.1 A standard communication simulation is a genuine transfer theorem

Suppose the input is partitioned as $x = (x_L, x_R)$ between two parties. A computation can be viewed as inducing an interactive protocol by giving one party the information associated with the left side and the other party the information associated with the right side. Every time the simulated computation requires information controlled by the other party, a message is exchanged. For a fixed computation of length T , the literal transcript therefore has length at most polynomially related to T (and, in a unit-bandwidth encoding, at most $O(T)$ bits of crossing information).

Thus there is a valid one-way transfer direction of the desired form:

A running for time $T \implies$ an induced protocol with communication $O(T)$.

This direction is not the missing theorem. It is essentially bookkeeping: a machine cannot communicate more information across an explicit simulated cut than the number of communication events it performs.

The mathematical difficulty lies in proving the reverse obstruction.

7.2.2 The communication ceiling

For an ordinary total two-party Boolean function $f : \{0, 1\}^N \rightarrow \{0, 1\}$, deterministic communication complexity satisfies

$$D(f) \leq N + 1,$$

because one party can simply send the entire input to the other party, after which the second party evaluates f . Variants of the bound differ by harmless additive or model-dependent terms, but the essential point is unchanged: standard communication complexity of an N -bit total function is at most linear in the input length.

Proposition 7.2.1 (Communication ceiling for the proposed route). *A lower bound on ordinary two-party deterministic communication complexity of a fixed N -bit function can never by itself be superpolynomial in N . Consequently, ordinary total-function communication complexity of a fixed input partition cannot directly yield a superpolynomial lower bound on the running time of a general polynomial-time SAT algorithm.*

Proof. The all-input transmission protocol gives $D(f) \leq N + 1$. Therefore any lower bound obtained solely in the ordinary deterministic two-party communication model is at most linear in N . Since SAT instances have input length N , such a lower bound cannot contradict polynomial time: linear communication is itself compatible with polynomial running time. To obtain a contradiction to $P = NP$, the communication lower bound would have to be superpolynomial in the input length, which the ordinary communication model forbids for total functions under the standard cost measure. $\square$ $\square$

This proposition does not make communication complexity irrelevant. It identifies a specific role in which it cannot serve as the entire final obstruction. Communication lower bounds can be extremely strong within restricted models, and lifting theorems can transfer query or communication hardness into circuit, proof, and related lower bounds. Such methods are an established part of modern lower-bound theory. [8, 9]

7.2.3 The trace-length dichotomy

The reviewer suggests measuring the amount of information “shuttled” across a partition. There are two natural interpretations, and both must be separated.

First, let r_{msg} count the number of bits sent in the literal execution transcript. Then

$$r_{\text{msg}}(E(A), x) \leq O(T_A(x)).$$

A superpolynomial lower bound on r_{msg} would indeed imply a superpolynomial time lower bound. But in this form the resource is simply a bounded portion of the execution time. Without an independent theorem showing that every fast solver must communicate superpolynomially, the resource has not escaped the tautology problem. The definition is useful for a simulation theorem but not yet explanatory.

Second, let r_{info} measure some compressed or semantic notion of information crossing the cut rather than literal transcript length. This is more promising because it could, in principle, be substantially smaller than time and therefore genuinely structural. But then the lower-bound theorem must prove that every correct SAT computation induces sufficiently large r_{info} under a suitable family of partitions. Establishing such a theorem is a new communication-complexity lower-bound problem, not an immediate consequence of the existence of the trace.

The distinction can be summarized as

literal transcript size $\Rightarrow$ valid but potentially tautological transfer,
semantic information measure $\Rightarrow$ potentially nontrivial, but new lower-bound problem.

7.2.4 Why a fixed partition is vulnerable to representation changes

A second attack concerns the partition itself. A polynomial-time solver may preprocess the formula, reorder variables, compute summaries, compress clauses, or transform the instance into a representation in which the proposed left/right separation no longer reflects the semantic structure of the problem. A lower bound tied to one literal partition of one representation therefore requires a transfer theorem showing that polynomial-time equivalent encodings preserve the relevant communication obstruction.

This issue is closely related to the reason lifting theorems are delicate. Existing lifting results normally begin with a carefully constructed composed function $f \circ g^n$, where the gadget g is chosen precisely so that a known query or decision-tree lower bound survives the passage to

communication complexity. They do not say that an arbitrary semantic problem automatically acquires large communication complexity from the mere existence of a complicated computation. [8]

Thus a future communication-based proof would need a structured encoding theorem of the form

$$\text{SAT instance} \longrightarrow \text{communication gadget instance} \longrightarrow \text{unavoidable lower bound},$$

with explicit control of the reduction and its parameters.

7.2.5 Dynamic traces and pebbling

The communication proposal also suggests a second resource: the graph of dependencies among configurations of a computation. This resembles pebbling and proof-complexity constructions, where one studies how much information must be retained or recomputed in order to reach a target configuration.

This route is mathematically richer than simply counting messages. It is also already close to an established lower-bound literature. Communication and pebbling viewpoints have been used to prove size-space and related tradeoffs for proof systems, and communication lower bounds have been connected to restricted proof complexity through explicit reductions.

However, the same quantifier problem reappears. A general polynomial-time Turing machine may recompute information, change representations, or use memory in ways that are not captured by a single fixed pebbling graph. Therefore a pebbling graph extracted from one computation provides a lower bound only after proving that the graph property is unavoidable for *every* computation realizing the same function.

Proposition 7.2.2 (Trace-graph tautology test). *Suppose a graph statistic $r(E(A), x)$ is defined solely from the explicit computation trace and satisfies $r(E(A), x) \geq cT_A(x)$ for some fixed $c > 0$. Then a lower bound on r is equivalent, up to constants, to a lower bound on T_A and therefore does not by itself provide an independent explanation of the hardness.*

Proof. The lower inequality gives $T_A(x) \leq r(E(A), x)/c$. Thus any asymptotic lower bound on r translates directly into the corresponding lower bound on the computation time, but the statistic contains no less information than the computation time it is intended to explain. The proposed resource is therefore a representation of runtime rather than an independently justified bottleneck. $\square$ $\square$

This proposition does not reject graph-based resources. It establishes an audit criterion: the useful case is where r is substantially more structural than raw trace length, while still admitting a one-way transfer theorem from computation to resource.

7.2.6 Current verdict on the dynamic communication trace

The reviewer proposal therefore survives in a refined form, but not in its strongest original form.

Component	Status	Current conclusion
Extraction of an execution trace	Valid construction	A computation can be represented as a trace or induced protocol.
Literal transcript-size transfer	Theorem-level bookkeeping	Communication is bounded by computation time.
Ordinary two-party communication as the final lower bound	Rejected route	Total N -bit functions have communication at most linear in N .
Semantic information across a cut	Research direction	Could be non-tautological, but requires a new universal lower-bound theorem.
Pebbling/dependency resource	Research direction with existing foundations	Promising for restricted models and tradeoffs; unrestricted transfer remains open.
Lifting from restricted communication to stronger models	Established methodology in selected settings	Requires a carefully constructed gadget and a theorem; not automatic for SAT.

The important consequence is positive as well as negative. The dynamic-trace idea has not merely been “criticized.” It has been decomposed into a valid transfer layer, a proved communication ceiling, a nontrivial semantic-resource subproblem, and a restricted-model route through lifting. That decomposition tells us exactly what a future communication-based proof would have to accomplish.

7.3 Meta-Complexity and Hardness Magnification as Candidate Bridges

The negative results above point more naturally toward meta-complexity. Instead of asking whether a witness itself is enormously complex, one asks whether recognizing, compressing, learning, or certifying the computational complexity of a represented object is hard. The Minimum Circuit Size Problem and related problems make this idea explicit.

Meta-complexity provides a possible bridge toward lower bounds rather than a solved route to $P \neq NP$.

Hardness magnification gives a second possible bridge. Atserias and Müller develop general magnification results in which modest lower bounds for suitable approximation problems can imply stronger circuit or formula lower bounds, including a uniform magnification result for MCSP. [11] This suggests an important strategic shift: a future proof need not begin by demanding an exponential lower bound. A weak but structurally meaningful lower bound attached to the residual family might, in principle, be amplified through an established theorem. The missing work is to construct the appropriate approximation or promise variant and verify that the relevant hypotheses actually apply.

This does not establish the required bridge for SAT. It does, however, provide a more technically grounded replacement for the vague instruction to “find an invariant.”

7.3.1 A residual meta-complexity problem

The semantic family suggests a concrete meta-complexity variant. Define ResMCSP as the circuit-size problem whose input is a Boolean formula φ together with a size bound s , asking

whether the semantic residual function $\widehat{F}_\varphi$ has a Boolean circuit of size at most s . The formula itself is the compact description of the semantic object; the residual function need not be expanded explicitly. This is not introduced as a hardness result. It makes the meta-complexity route intrinsic to the present construction rather than importing MCSP only by analogy.

The immediate questions are structural: identify trivial parameter regimes, determine which reductions preserve residual semantics, compare uniform and nonuniform formulations, and locate the problem relative to established circuit-size and partial-function complexity problems. The purpose is to obtain a separately attackable object whose complexity behavior may inform the search for a structural certificate for the residual family.

7.3.2 Gate elimination as a general-circuit lower-bound laboratory

General Boolean circuit lower bounds remain the central obstacle. Recent work has made gate elimination especially relevant to this programme: it formalizes circuit simplification as a convergent rewriting process in several common bases and derives constructive lower bounds, while related work extracts efficient refuters from gate-elimination arguments. [20, 21] The relevance here is methodological. A convergent simplification process supplies a canonical notion of what information survives local circuit elimination, reducing dependence on the order in which simplifications are applied.

A concrete residual experiment is therefore possible: choose a circuit basis, define a semantics-preserving simplification system, apply it to candidate circuits for $\widehat{F}_m$, and ask whether every sufficiently small circuit is forced into a normal form incompatible with a residual-family property. No such theorem is claimed here. The point is that this creates a testable route from the semantic object to an established unrestricted-circuit lower-bound methodology rather than leaving “general circuit lower bound” as a single undifferentiated obstacle. [20]

7.4 A More Precise Meaning of Accessibility

The word “accessibility” should not mean that a solution space merely looks complicated. It should mean that every computation in a specified resource model must cross a certified bottleneck. In a general computation model, this suggests the following route:

$$A \in P \implies E(A) \in \mathcal{M} \implies r(E(A), x) \leq \text{poly}(T_A(x), |x|),$$

followed by an independent theorem forcing $r(E(A), x)$ to be superpolynomial on adversarial inputs.

This formulation is stronger than a scalar “difficulty” score and weaker than simply defining r to be the running time. It asks for a genuine resource theory with a transfer theorem.

7.5 The Universal Extraction Lemma

Conjecture 7.5.1 (Universal extraction lemma). *There exists a resource model $\mathcal{M}$, an NP-complete self-reducible search relation R , an extraction map E , and a resource functional r such that:*

(a) *every deterministic polynomial-time solver A for R yields $E(A) \in \mathcal{M}$ with*

$$r(E(A), x) \leq \text{poly}(T_A(x), |x|);$$

(b) for every polynomial-time solver A there are arbitrarily large inputs x with

$$r(E(A), x) > |x|^c$$

for every fixed constant c ;

(c) the resource lower bound is stable under the polynomial reductions needed to transfer the result to SAT; and

(d) the proof uses non-black-box structure and is not merely an instance of a known relativizing, natural-proofs, or algebrizing technique.

Proposition 7.5.2 (The universal extraction lemma would imply $P \neq NP$). *If the universal extraction lemma is proved, then $P \neq NP$.*

Proof. This is the resource-transfer principle. Under $P = NP$, SAT has a polynomial-time solver. The extracted resource is then polynomially bounded by the transfer theorem, while the adversarial lower-bound clause forces it to be superpolynomial on arbitrarily large inputs. Contradiction. $\square$ $\square$

The importance of this statement is not that it is already true, but that it decomposes the previous placeholder into a falsifiable mathematical object. A proposed extraction map can be attacked for non-uniformity, circularity, representation dependence, or lack of transfer; a proposed resource can be attacked for algorithmic bypass; and a proposed lower bound can be attacked independently.

7.6 Adversarial Research Ladder

Every future candidate should pass the following sequence in order:

definition $\rightarrow$ restricted-model lower bound $\rightarrow$ representation stability
 $\rightarrow$ resource extraction $\rightarrow$ adversarial largeness
 $\rightarrow$ barrier audit $\rightarrow$ NP-complete transfer.

A failure at stage k does not erase stages $1, \dots, k-1$. It determines the exact reason the proposed route cannot yet reach unrestricted computation. This is the intended convergence mechanism of the research programme.

7.7 Why Brute-Force Explosion Is Not Yet a Lower Bound

A common argument says: SAT has 2^n possible assignments, so any algorithm must in some sense examine exponentially many possibilities. That statement is false as a universal theorem. An algorithm need not enumerate the search space.

For some structured SAT instances, clever algorithms prune enormous portions of the space. More generally, a small description can encode a huge combinatorial search while admitting an efficient dynamic-programming or algebraic shortcut.

The existence of an exponentially large search space therefore proves only that *naïve enumeration* is exponential. It does not prove that every algorithm is exponential.

Proposition 7.7.1 (Search-space cardinality does not imply time lower bound). *For a problem family with $2^{p(n)}$ candidate witnesses, candidate-space cardinality alone does not imply an $\Omega(2^{p(n)})$ lower bound for every deterministic algorithm.*

Proof. The number of possible witnesses is a property of the representation, not a lower bound on the amount of computation needed to determine existence. An algorithm may exploit algebraic, combinatorial, recursive, or other structure to summarize exponentially many candidates. Therefore a lower bound requires an argument that every relevant algorithm must expend sufficient resources, not merely that the raw witness space is large. $\square$ $\square$

This proposition is elementary but indispensable. A rigorous proof of P versus NP must rule out the “hidden shortcut” possibility.

7.8 Cryptography: Powerful Evidence, Not a P versus NP Proof

The original conversation repeatedly connected P versus NP to one-way functions and cryptography. There is a genuine connection, but it must be stated carefully.

If $P = NP$, then standard NP-complete search and decision problems become polynomial-time solvable, which would have profound consequences for many complexity-based cryptographic assumptions. However, the security of particular cryptosystems such as RSA is not equivalent to $P \neq NP$. Factoring is not known to be NP-complete, and the collapse $P = NP$ would destroy many conventional assumptions, but the converse implication from the observed security of a particular cryptosystem to $P \neq NP$ is not available.

The Natural Proofs literature makes this connection especially relevant: standard cryptographic hardness assumptions can themselves obstruct broad lower-bound proof methods. [4]

Thus cryptography is best treated as a consequence domain and a source of methodological evidence, not as a substitute for the missing worst-case lower bound.

7.9 The “If It Were P, Someone Would Have Found It” Argument

A more sociological form of the argument is:

P versus NP has been studied for decades by enormous communities of mathematicians and computer scientists. No polynomial-time algorithm has appeared. Therefore $P \neq NP$ is overwhelmingly likely.

This is a reasonable Bayesian belief about the world. It is not a theorem of classical complexity theory.

One cannot convert the absence of discovery into a universal lower bound unless one proves a theorem connecting the research process to the algorithmic search space. Human research effort is not itself a well-defined complexity class, and failure to find an algorithm does not imply that one does not exist.

Nevertheless, the social observation has scientific value. It can motivate hypotheses about which structures are difficult to exploit. It can also identify families of approaches that repeatedly fail. But it should remain in the motivation and strategy sections of a paper, not in the proof core.

The Repaired Research Program

8.1 A Constructive Search-to-Decision Framework

The strongest concrete mathematical object to emerge from the verification-generation idea is a two-way interface among three tasks:

decision, search, verification.

For self-reducible NP-complete problems, these tasks are polynomially equivalent in an important sense.

Theorem 8.1.1 (Decision–search equivalence for SAT). *For Boolean satisfiability, decision and total witness construction are polynomial-time equivalent, while verification of a proposed satisfying assignment is polynomial-time.*

Proof. Verification is polynomial-time by direct evaluation of the formula. Decision $\Rightarrow$ search is the search-to-decision reconstruction theorem. Conversely, a total certifying witness generator decides SAT by accepting exactly when it outputs a valid satisfying assignment rather than failure. Thus SAT decision and total witness construction are polynomial-time equivalent. $\square$ $\square$

The theorem does something important for the present project: it validates the intuitive focus on generation without changing the formal problem. A proof that generation requires superpolynomial time for SAT would be a proof of $P \neq NP$.

8.2 A Stronger Adversarial Program

Instead of trying to argue globally that “search is hard,” we can ask a more controlled sequence of questions.

8.2.1 Question A: Can we lower-bound generators in a restricted model?

This is often possible and has a large literature. Examples include decision-tree lower bounds, communication complexity, circuit complexity, proof complexity, branching-program lower bounds, and query lower bounds. Such theorems establish real hardness but usually for a restricted resource model.

8.2.2 Question B: Can the restricted model approximate general polynomial time?

This is harder. A restricted-model lower bound settles P versus NP only if one proves that every polynomial-time algorithm can be represented or reduced to the restricted model without losing the lower bound. This is precisely where many otherwise strong approaches fail.

8.2.3 Question C: Can an algorithm-to-lower-bound conversion be proved?

Williams' work on circuit lower bounds demonstrates a striking version of an algorithm-to-lower-bound connection: faster algorithms for restricted circuit-satisfiability problems can be converted into circuit lower bounds. [6]

This suggests a fruitful methodological inversion:

to prove that no fast algorithm exists, study what a fast algorithm would imply structurally.

That principle may be more promising than trying to inspect all algorithms directly.

8.3 An Explicit Audit of Common Failed Proof Patterns

This section records proof patterns that appear persuasive but are invalid as general proofs unless supplemented with additional lemmas.

8.3.1 Pattern 1: “There are exponentially many possibilities.”

Failure: candidate count is not an algorithmic lower bound.

Required repair: prove that every algorithm must extract enough information to distinguish the relevant cases, via a genuine model-independent lower-bound mechanism.

8.3.2 Pattern 2: “Verification is polynomial, so search is harder.”

Failure: this is exactly the distinction whose universality is under question.

Required repair: prove a superpolynomial lower bound on search or decision for a specific NP-complete language.

8.3.3 Pattern 3: “Many AIs cannot find a proof.”

Failure: finite families of heuristic systems do not quantify over all polynomial-time algorithms.

Required repair: use the systems only to generate conjectures, identify counterexamples, or discover invariants; derive the final lower bound independently.

8.3.4 Pattern 4: “Gödel prevents the system from proving itself.”

Failure: incompleteness is not equivalent to time complexity separation.

Required repair: provide a precise reduction from the relevant incompleteness statement to a superpolynomial lower bound, with all formal-system hypotheses stated.

8.3.5 Pattern 5: “No one has found the shortcut.”

Failure: absence of discovery is empirical evidence, not universal impossibility.

Required repair: prove a theorem about all possible polynomial-time algorithms or adopt an explicit probabilistic/model-dependent claim and label it as such.

8.3.6 Pattern 6: “Peer review eventually accepts the proof.”

Failure: community consensus is evidence of reliability, not itself a formal derivation.

Required repair: separate theoremhood in the formal system from the sociological process by which humans become confident that the formal object has been correctly read.

Hard-Coding, Reduction Stability, and Restricted Models

9.1 The Central Attack: The Original Universal Γ Cannot Exist

The earlier formulation introduced a “universal accessibility lower-bound functional”

$$\Gamma(x)$$

with the requirement that every correct algorithm A solving the relevant search problem satisfy

$$T_A(x) \geq \Gamma(x)$$

for the same individual instance x . The quantifier structure looked attractive because it appeared to turn an intrinsic property of the instance into a universal lower bound.

There is, however, a decisive obstruction. The pointwise universal statement is too strong even before one asks whether Γ is computable, geometric, topological, information-theoretic, or barrier-resistant.

9.1.1 The hard-coding theorem

We first state the obstruction in its most elementary form.

Theorem 9.1.1 (Pointwise universal lower bounds are destroyed by hard-coding). *Let $R(x, w)$ be a polynomially balanced search relation, and suppose there exists at least one total deterministic algorithm B that correctly solves its search problem: on every yes-instance it outputs some w satisfying $R(x, w)$, and on every no-instance it correctly reports that no witness exists. Fix an input x .*

Then there exists another total correct deterministic algorithm B_x which agrees with B on every input except that it hard-codes the correct answer for x . Consequently, on the ordinary sequential input model,

$$T_{B_x}(x) = O(|x| + |w_x|) = |x|^{O(1)},$$

with the witness length polynomially bounded in $|x|$. In a unit-cost RAM presentation in which the distinguished instance and its witness are treated as preloaded constants, the special-case dispatch cost can even be $O(1)$, although outputting a nonconstant witness still requires time proportional to its length.

Therefore no functional $\Gamma(x)$ satisfying

$$T_A(x) \geq \Gamma(x)$$

for every correct algorithm A can be superpolynomial on every sufficiently large instance x for which a correct algorithm exists. The hard-coded solver still has to output a witness of polynomially bounded length, so the argument rules out superpolynomial growth, not every polynomially superlinear bound.

Proof. Fix x .

If x is a yes-instance, choose one valid witness w_x and define B_x as follows. On input y , first compare y with the hard-coded string x . If $y = x$, output the hard-coded witness w_x . Otherwise run $B(y)$. If x is a no-instance, the same construction hard-codes the correct “no witness” response at the exceptional input.

The comparison requires at most $O(|x|)$ symbol comparisons on the standard sequential model, and the special-case output requires at most polynomially many steps in $|x|$ because the witness length is polynomially bounded. On every other input the machine behaves exactly like B . Hence B_x is a total correct solver for the same search problem, while its running time on the particular instance x is polynomial in $|x|$.

Now suppose a universal pointwise lower bound $\Gamma(x)$ were superpolynomial on a sequence of instances. Applying the construction to each such fixed instance gives a correct algorithm whose running time on that instance is polynomial in its length, contradicting the proposed inequality once $\Gamma(x)$ exceeds every polynomial.

The contradiction uses only the existence of one correct solver and the freedom to choose a different correct algorithm for each fixed instance. It does not depend on relativization, natural proofs, algebrization, cryptography, or any philosophical notion of proof. $\square$ $\square$

Remark 9.1.2 (Why this is a useful failure). *This result does not attack the desired conclusion $P \neq NP$. It attacks one incorrect quantifier order that attempted to express the conclusion.*

The problematic form was

$$\forall x \forall A \in \mathcal{A}_{\text{correct}} T_A(x) \geq \Gamma(x).$$

The hard-coding construction shows why a worst-case separation cannot be expressed in this pointwise form.

The mathematically viable lower-bound forms move the existential quantifier over hard instances after the algorithm has been fixed:

$$\forall A \exists x \quad T_A(x) \text{ is large,}$$

or, for a polynomial-time class indexed by a clock bound,

$$\forall k \forall A \in \text{DTIME}(n^k) \exists x \quad T_A(x) > n^k.$$

This is precisely the quantifier pattern that a genuine $P \neq NP$ proof must ultimately defeat.

9.1.2 A stronger version: finite exceptional sets are never enough

The preceding theorem is not merely a warning about constants. Any fixed finite list of exceptional instances can be hard-coded into a correct algorithm.

Proposition 9.1.3 (Finite-exception elimination). *Let F be a finite set of inputs for a total decidable language L . If B is a correct decider for L , then there is a correct decider B_F that agrees with B outside F and answers every input in F by a hard-coded table. Thus any purported lower bound against all algorithms must be asymptotic and must allow the hard instances to escape every finite exceptional set.*

Proof. Store the finite table

$$\{(x, L(x)) : x \in F\}$$

in the description of B_F . On input y , compare y against each stored string. If a match occurs, output the stored answer; otherwise run $B(y)$. Since F is finite, the additional work is a fixed amount of program-description-dependent computation, and correctness is immediate. $\square$ $\square$

The proposition gives a useful design principle:

hardness against all polynomial algorithms must persist beyond every finite collection of individually protected inputs.

That is an asymptotic requirement, not a rhetorical one.

9.1.3 Repair: make the obstruction algorithm-relative

The failed pointwise invariant should therefore not be discarded completely. It should be changed from an instance-only object into an *algorithm-relative certificate*.

Definition 9.1.4 (Algorithm-relative accessibility certificate). *Fix a search relation R . A function*

$$\Gamma(A, x) \in \mathbb{R}_{\geq 0}$$

is an algorithm-relative lower-bound certificate if it satisfies

$$R \text{ is correctly solved by } A \implies T_A(x) \geq \Gamma(A, x)$$

for every admissible algorithm A and every input x .

The distinction is fundamental. The hard-coding algorithm B_x may make its own certificate $\Gamma(B_x, x)$ small, and that is exactly what should happen. A valid lower-bound method must therefore explain *why the algorithm cannot avoid paying* on some inputs, rather than merely assigning an intrinsic hardness value to each input.

This change transforms the research problem from

“find a hard instance”

into

find a sound adversarial certificate that defeats every polynomial algorithm on some instance

That formulation is much closer to established lower-bound methodology.

9.1.4 The exact repaired universal lemma

We can now state the lemma that would actually suffice for $P \neq NP$.

Conjecture 9.1.5 (Quantifier-correct universal generator lower bound). *There exists a polynomial-time self-reducible NP-complete search relation R and a sound algorithm-relative certificate $\Gamma(A, x)$ such that for every deterministic polynomial-time algorithm A that correctly solves R , there are arbitrarily large input lengths n and inputs x of length n for which*

$$\Gamma(A, x) > n^c$$

for every fixed constant c .

This statement is deliberately stronger in quantifier discipline and weaker in instance uniformity than the previous $\Gamma(x)$ definition. It no longer contradicts the hard-coding theorem because A is fixed before the hard instance is selected.

Proposition 9.1.6 (The repaired lemma would settle $P \neq NP$). *If the preceding conjecture is proved, then $P \neq NP$.*

Proof. Suppose $P = NP$. Since R is NP-complete and polynomially self-reducible, the standard search-to-decision reconstruction gives a polynomial-time correct search algorithm A for R . By the conjecture, for every constant c there are arbitrarily large inputs x on which

$$\Gamma(A, x) > |x|^c.$$

Soundness of Γ gives

$$T_A(x) \geq \Gamma(A, x) > |x|^c$$

for arbitrarily large x . This contradicts the assumption that A runs in polynomial time. Hence $P \neq NP$. $\square$

The proposition is a genuine theorem. The conjecture is the research target. The two statements must not be conflated.

9.1.5 What an algorithm-relative certificate might look like

The revised object should ideally combine four ingredients:

- (i) *semantic content*: it measures a requirement imposed by the language or search relation rather than by arbitrary syntax;
- (ii) *algorithm sensitivity*: it depends on what the proposed algorithm can observe, remember, query, or compress;
- (iii) *lower-bound soundness*: every successful computation must pay at least the certified amount;
- (iv) *reduction stability*: polynomial-time translations cannot erase the obstruction without paying only polynomial overhead.

Decision-tree adversary arguments satisfy an analogue of these requirements in a very restricted model. Circuit lower bounds, communication lower bounds, proof-complexity arguments, and time-space tradeoffs each implement related ideas in different resource models. The open problem is to obtain a sufficiently strong construction that applies to unrestricted polynomial-time computation.

9.1.6 Why certificate size, witness count, and naive geometry are not enough

Several tempting choices can be rejected by explicit counterexamples.

First, the number of witnesses does not control search time. For any efficiently decidable language L , introduce irrelevant witness bits:

$$R'(x, w, z) \iff R(x, w), \quad z \in \{0, 1\}^{q(|x|)}.$$

Then each valid witness can be multiplied into $2^{q(|x|)}$ witnesses without changing the underlying search task in any essential way: a solver can ignore z .

Second, a unique witness need not be computationally hard. An input may encode an easily extractable canonical witness, so

$$|W_x| = 1$$

has no implication about worst-case running time.

Third, a geometric quantity defined on a particular representation is not automatically a semantic invariant. A polynomial-time reduction can introduce auxiliary variables, duplicate constraints, reorder variables, encode the same object by gadgets, or add redundant structure. Any quantity used for a universal lower bound must come with a transfer theorem showing that these transformations preserve the lower-bound meaning.

Proposition 9.1.7 (Witness padding defeats cardinality-based lower bounds). *No function of the witness-set cardinality alone can yield a universal superpolynomial search lower bound for polynomially verifiable relations.*

Proof. Given any polynomially verifiable relation $R(x, w)$, define

$$R_q(x, (w, z)) \iff R(x, w)$$

with $z \in \{0, 1\}^{q(|x|)}$. A witness for R can be converted to a witness for R_q by appending any z , and a witness for R_q can be projected to a witness for R . Both directions are computable in polynomial time. Yet

$$|W_x^{(q)}| = |W_x| 2^{q(|x|)}.$$

Thus witness multiplicity can be changed exponentially while preserving the essential search problem. A cardinality-only lower bound would therefore depend on a representational choice rather than on the computational obstruction being sought. $\square$ $\square$

9.1.7 A useful reinterpretation of “accessibility”

The word “accessibility” can now be retained, but its meaning must change.

It should not mean

“the solution set looks difficult”.

It should mean

every computation in the specified model must cross a certified
computational bottleneck before it can decide or produce a valid witness.

That is a much narrower statement. It is also a statement for which established lower-bound tools exist in restricted settings.

9.2 Reduction Robustness

If a lower-bound quantity is to prove P versus NP through SAT, it must behave well under polynomial-time reductions.

Suppose L is NP-complete and $L' \leq_p L$. A polynomial-time algorithm for L yields one for L' . Therefore, if $L' \notin P$, then $L \notin P$.

For search measures, the situation is more delicate. A reduction may transform witnesses in a nontrivial way or increase witness length. Therefore a candidate search invariant should be defined so that it survives polynomial-time preprocessing and postprocessing with only polynomial distortion.

Proposition 9.2.1 (Necessary reduction stability). *Any resource measure intended to imply $P \neq NP$ through NP-completeness must be monotone enough under polynomial-time reductions that a polynomial algorithm for SAT would contradict the lower bound on the target language.*

Proof. Otherwise an alleged lower bound on a target NP language could coexist with a polynomial reduction from that language to SAT without contradicting the assumption $SAT \in P$. Hence the measure would not be a valid obstruction to P versus NP. $\square$ $\square$

This is a useful audit criterion for future work.

9.3 Sensitivity and Topology: Productive Experiments, Not Automatic Lower Bounds

Two suggestions deserve a careful status distinction.

9.3.1 Boolean sensitivity

Huang proved the Sensitivity Conjecture: classical Boolean-function sensitivity is polynomially related to several other measures, thereby resolving a long-standing structural question about Boolean functions. [7] This is a genuine example of a combinatorial measure becoming substantially more powerful than its original appearance suggested.

However, the theorem does not imply a superpolynomial lower bound for SAT against arbitrary polynomial-time algorithms. Sensitivity is a property of a Boolean function, while P versus NP asks for a universal time lower bound over algorithms computing an NP-complete function. A bridge between the two would require, at minimum, a theorem of the schematic form

$$\text{Sens}(f_n) \geq h(n) \implies T_A(f_n) \geq g(n)$$

for every admissible algorithm A in a class broad enough to capture polynomial-time computation. No such theorem is asserted here.

Thus sensitivity is retained as a legitimate research resource, but its present status in this paper is:

known structural theorem + open transfer problem.

9.3.2 Topology of witness spaces

Likewise, one may construct a simplicial complex or other topological object from the solution space of a SAT instance, calculate connected components or homology groups, and search for correlations between these quantities and solver behavior. This can be scientifically useful for discovering instance families and falsifying simplistic structural hypotheses.

But an observed correlation is not a worst-case lower bound. To turn topology into a proof one would need:

- (a) a representation-independent topological object;
- (b) a theorem that polynomial-time reductions preserve the relevant quantity up to controlled distortion;
- (c) a theorem connecting the quantity to the resource usage of *every* polynomial-time algorithm, including algorithms that do not explicitly traverse the represented solution space;

- (d) a proof that the hard-instance family has asymptotically large invariant.

Condition (c) is the decisive difficulty. An algorithm may exploit algebraic structure, compression, symmetry, dynamic programming, preprocessing, or an entirely different representation. Therefore “the solution space is topologically complicated” cannot itself imply “every algorithm must spend superpolynomial time.”

The experimental program is nevertheless retained, because it can serve the logically appropriate role of a *conjecture generator* and *counterexample engine*.

9.4 A Concrete Restricted Sandbox: What the Invariant Can Actually Prove

The reviewer asked for a restricted computational environment in which the proposed notion of accessibility can be made concrete rather than remaining a placeholder. We now do exactly that.

The purpose of the sandbox is not to claim that a restricted-model theorem solves P versus NP. Its purpose is to determine which parts of the intuition can survive formalization and which parts disappear as soon as the model is enlarged.

9.4.1 Decision trees

Let

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

be a Boolean function. A deterministic decision tree algorithm queries input bits and must determine $f(x)$ on every input.

Let $\text{DT}(f)$ denote its deterministic decision-tree complexity, the minimum worst-case number of bit queries required by any decision tree computing f .

Definition 9.4.1 (Block-sensitivity certificate). *For $x \in \{0, 1\}^n$, let $\text{bs}_x(f)$ be the maximum number of pairwise disjoint blocks of coordinates*

$$B_1, \dots, B_k$$

such that flipping all bits in any one block changes the value of f . Define

$$\text{bs}(f) = \max_x \text{bs}_x(f).$$

Theorem 9.4.2 (Block sensitivity is a sound decision-tree lower bound). *For every Boolean function f ,*

$$\text{DT}(f) \geq \text{bs}(f).$$

Proof. Choose x attaining $\text{bs}(f) = k$, with pairwise disjoint sensitive blocks $B_1, \dots, B_k$.

Consider any deterministic decision tree computing f . For each block B_i , if the tree never queries any coordinate in B_i along the computation path reached by x , then the input obtained by flipping all bits in B_i produces the same observed query answers and therefore follows the same path to the same output. But by sensitivity, that flipped input has the opposite function value, a contradiction.

Hence the computation path on x must contain at least one query from every block B_i . Since the blocks are disjoint, these are at least k distinct queries. Therefore every decision tree has a worst-case path of length at least k , proving

$$\text{DT}(f) \geq k = \text{bs}(f).$$

□

□

This is a genuine algorithm-relative lower-bound mechanism in a restricted model. It has an adversarial interpretation: the sensitive blocks are independent places at which an algorithm must acquire information.

The result also exposes the limitation immediately:

$$\text{bs}(f) \leq n.$$

Thus this certificate cannot by itself produce superpolynomial lower bounds in the standard n -bit decision-tree model.

9.4.2 Sensitivity as a diagnostic rather than a universal obstruction

Huang proved the Sensitivity Conjecture, showing that sensitivity and block sensitivity are polynomially related. [7] This is a major theorem about Boolean function complexity, but it does not imply P versus NP. The reason is structural: decision-tree complexity measures unrestricted access to individual input bits, while a polynomial-time Turing machine or circuit may compute with gates that aggregate, reuse, transform, and compress information.

Consequently a sensitivity-derived accessibility invariant can be powerful in a query model without being a universal time lower bound.

This is precisely the kind of distinction the research program needs to preserve:

a strong lower bound in model $M \not\Rightarrow$ the same lower bound in all models.

9.4.3 Monotone circuits: a second sandbox

A more expressive restricted model is monotone Boolean circuits. Here the input variables represent, for example, edges of a graph, and circuits may use AND and OR gates but no NOT gates.

The CLIQUE function is monotone in the graph edges. Strong lower bounds are known in this model.

Imported Theorem 9.4.3 (Monotone circuit lower bounds for CLIQUE). *There are explicit ranges of $k = k(n)$ for which the monotone circuit complexity of $\text{CLIQUE}_{k,n}$ is*

$$n^{\Omega(k)},$$

and recent refinements establish bounds of the form $n^{\Omega(\delta^2 k)}$ for appropriate

$$k \leq n^{1/3-\delta}.$$

In particular, the monotone complexity can be superpolynomial or exponential in the relevant parameters.

This theorem is an imported result from the monotone-circuit literature; we do not re-prove it here. The important point for the present program is methodological: there really are settings in which a combinatorial obstruction can be converted into a superpolynomial computational lower bound.

Remark 9.4.4 (Why this does not prove $P \neq NP$). *CLIQUE is NP-complete under ordinary polynomial-time reductions, but the theorem concerns monotone circuits rather than general polynomial-time computation. A general algorithm can use negations, arithmetic encodings, dynamic programming, inclusion–exclusion, algebraic transforms, or other mechanisms unavailable to the monotone model.*

Therefore the implication

$$\text{monotone CLIQUE lower bound} \implies \text{CLIQUE} \notin P$$

is invalid.

The missing transfer theorem would have to show that every polynomial-time method for CLIQUE can be converted, with controlled overhead, into a monotone circuit of the same restricted kind. No such theorem is available, and it would be a revolutionary result if it were strong enough to settle P versus NP .

9.4.4 The sandbox ladder

The two restricted models suggest a hierarchy:

$$\begin{array}{ccccc} \text{decision tree} & \longrightarrow & \text{monotone circuit} & \longrightarrow & \text{general circuit/Turing computation} \\ \text{explicit adversary} & & \text{structural lower bound} & & \text{unrestricted lower bound.} \end{array}$$

The first arrow enlarges the algorithmic model while preserving enough structure to support strong lower bounds. The second arrow is the dangerous one. Any putative universal invariant must survive the removal of the syntactic restrictions.

This suggests a concrete refinement rule:

Every candidate invariant should be tested at the largest model for which its lower-bound proof remains valid, and the first model at which the proof breaks should be recorded explicitly.

That rule prevents a restricted-model theorem from being silently promoted into a P-versus-NP theorem.

9.4.5 Track C: algorithm-to-lower-bound transference

A second concrete route is to reverse the direction of the desired invariant.

Instead of beginning with a proposed hardness measure and asking whether every algorithm must pay it, begin with a restricted algorithmic problem. Suppose a circuit class $\mathcal{C}$ admits an algorithm for its satisfiability problem that is meaningfully faster than exhaustive search. Williams’ work demonstrates that for important classes, such SAT algorithms can be converted into circuit lower bounds; his ACC result is a canonical example. [6]

The structural pattern is

$\text{nontrivial algorithm for SAT}(\mathcal{C}) \implies \text{lower bound against } \mathcal{C}.$
--

This is not merely a philosophical analogy. It is a proved methodology in the appropriate circuit classes.

Imported Theorem 9.4.5 (Williams’ ACC transference result). *There are explicit versions of the following phenomenon: nontrivial satisfiability algorithms for suitable circuit classes can imply strong nonuniform circuit lower bounds. In particular, Williams proved that NEXP does not have polynomial-size ACC circuits, using faster algorithms for ACC satisfiability as a central ingredient.*

The significance for the present project is that it supplies a concrete research architecture for Γ : rather than guessing an intrinsic geometry of SAT solutions, seek a restricted computational class in which a structural feature admits both an algorithmic exploitation and a lower-bound interpretation.

The remaining question is whether this transference can be strengthened far enough to reach general polynomial-time computation. That step is precisely where current barriers remain.

9.4.6 Meta-complexity as a possible bridge

A particularly relevant modern direction is meta-complexity: problems whose inputs are themselves objects describing computational complexity, such as the Minimum Circuit Size Problem (MCSP). This is conceptually close to the repaired $\Gamma(A, x)$ idea. Rather than trying to assign an intrinsic “difficulty” to an instance with no reference to the algorithm, meta-complexity asks questions in which computational resources are themselves part of the mathematical object.

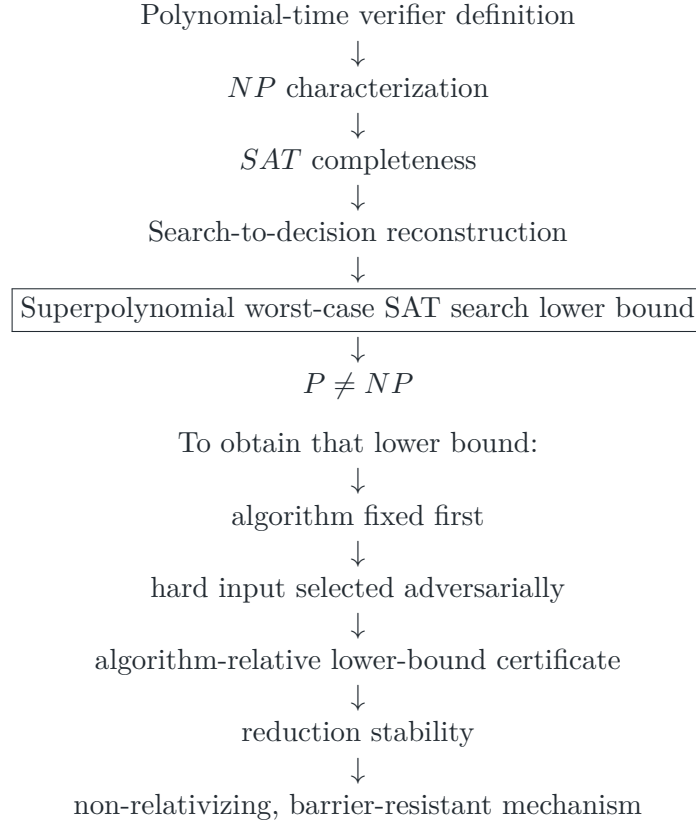
We therefore add a fourth route:

algorithm-relative obstruction $\longrightarrow$ meta-complexity object $\longrightarrow$ lower-bound transfer.

This is a research direction, not a theorem of the present paper.

9.5 A Proof Dependency Map

The logical dependency structure of the present program can be displayed as follows.



The present programme proves the first four levels and identifies the unresolved lower-bound step.

That is not a rhetorical retreat. It is the exact mathematical frontier reached by the argument in its present form.

9.6 Could the Core Argument Still Be Saved?

Yes, but only through additional mathematics.

The original insight can be sharpened in at least four directions.

9.6.1 Direction I: Information-theoretic obstruction

One may try to prove that a valid solver must obtain a quantity of information that cannot be extracted in polynomially many steps. This would require a rigorous model connecting information requirements to computational time. Shannon-style counting alone is insufficient because computational descriptions may exploit structure.

9.6.2 Direction II: Proof-complexity obstruction

One may construct explicit unsatisfiable formulas whose shortest proofs in every sufficiently broad proof system are superpolynomial. A fully universal result here would be decisive, but current theory shows why this is difficult. Lower bounds for restricted systems are substantial but do not yet rule out arbitrary polynomially bounded proof systems. [10]

9.6.3 Direction III: Circuit lower bounds

Because polynomial-time computation can be related to polynomial-size circuit families under standard uniformity transformations, sufficiently strong circuit lower bounds could imply $P \neq NP$. The difficulty is that general circuit lower bounds are themselves a major frontier. The natural-proofs and algebrization barriers explain why many broad strategies fail. [4, 5] Recent work on convergent gate elimination provides a useful contemporary laboratory for explicit unrestricted-circuit lower bounds, although it does not remove the central asymptotic barrier faced here. [20, 21]

9.6.4 Direction IV: Algorithm-lower-bound duality

An especially attractive strategy is to identify a restricted algorithmic problem such that any sufficiently fast solver would imply an impossible circuit or proof-system consequence. This is the spirit of several modern lower-bound connections, including work of Williams. [6]

These directions retain the original intuition but move it into formal lower-bound machinery.

9.7 What the Hard-Coding Attack Has Established

The most important unconditional result here is not a separation of complexity classes. It is a structural theorem about how such a separation cannot be formulated.

Theorem 9.7.1 (Quantifier obstruction for instance-only accessibility). *Any lower-bound program that assigns a superpolynomial hardness value to individual instances and claims that the value is a lower bound for every correct algorithm on that same instance is false in the standard uniform algorithmic setting.*

This theorem is simply the hard-coding theorem restated at the level of methodology, but that restatement is useful because it permanently removes one entire family of possible arguments.

The work therefore converges to the following corrected architecture:

$$\boxed{\forall A \in \mathcal{P} \quad \exists^\infty n \quad \exists x \in \{0, 1\}^n \quad \Gamma(A, x) > n^c}$$

for every fixed polynomial exponent c , with Γ independently characterized and sound.

The order

$$A \longrightarrow x$$

is not cosmetic. It is mathematically forced by the hard-coding counterexample.

9.7.1 An explicit rule against self-defeating invariants

A candidate invariant is rejected immediately if it satisfies only

$$\Gamma(x) \approx \min_A T_A(x),$$

because the quantity on the right is simply the unknown computational hardness of the instance under the chosen model.

Likewise, a candidate is rejected if its proof of largeness assumes, in disguised form, the nonexistence of a polynomial-time algorithm. Such a definition is circular.

The acceptable target is instead a theorem of the form

$$\Gamma(A, x) \leq T_A(x)$$

whose left-hand side is independently analyzable and whose largeness can be proved for an adversarially selected input after A is fixed.

This gives the project a falsifiable criterion for progress: either a proposed Γ earns the inequality by proof, or it does not.

9.8 What Would Count as a Successful New Result?

The project should not use a binary metric of “solved” versus “worthless.” There is a hierarchy of genuine mathematical achievements.

1. A new invariant with exact definitions.
2. An unconditional lower bound in a nontrivial computational model.
3. A reduction showing that the invariant is preserved under polynomial-time transformations.
4. A theorem that the invariant applies to an NP-complete language.
5. A proof that the invariant escapes known barriers.
6. A universal lower bound for all polynomial-time algorithms.

Level 6 settles P versus NP. Levels 1–5 can nevertheless represent substantial research progress if each is proved correctly and improves on known machinery.

9.9 A Carefully Stated Main Conjecture for the Research Program

The following statement captures the intended ambition without pretending that the hard step has already been established.

Conjecture 9.9.1 (Verification-to-generation hardness principle, repaired). *There exists a polynomial-time verifiable NP-complete relation R and a reduction-stable complexity invariant Γ such that:*

- (a) $\Gamma(A, x)$ is computable or lower-bounded by a finite, explicit certificate;
- (b) every polynomial-time verifier for R has bounded verification cost;
- (c) after each deterministic polynomial-time certifying generator A is fixed, the adversarially chosen hard-instance family forces $\Gamma(A, x)$ above every polynomial threshold infinitely often;
- (d) the proof of the lower bound on Γ is non-relativizing and escapes the standard natural-proof and algebrization barriers.

Then $P \neq NP$.

Remark 9.9.2. *This is a research target rather than a disguised conclusion. The logical implication is straightforward; the novelty would reside entirely in constructing Γ and proving the universal lower bound.*

The Surviving Mathematical Core

10.1 The Strongest Unconditional Conclusions of the Audited Program

We can now state the results that survive the adversarial audit.

Theorem 10.1.1 (Surviving mathematical core). *The following statements hold unconditionally within the standard deterministic Turing-machine framework:*

- (a) *NP admits a polynomial-time verifier characterization.*
- (b) *For SAT, polynomial-time decision implies polynomial-time witness construction.*
- (c) *Therefore, any superpolynomial lower bound on SAT search time for all deterministic polynomial-time generators would imply $P \neq NP$.*
- (d) *Fixed finite families of efficient auditors do not, by themselves, yield a P versus NP separation.*
- (e) *Finite computational testing cannot establish an unrestricted universal complexity lower bound without an additional structural theorem.*
- (f) *Gödel incompleteness, by itself, does not imply $P \neq NP$.*
- (g) *The existence of a large candidate space, by itself, does not imply an exponential lower bound for all algorithms.*
- (h) *Any universal proof of $P \neq NP$ must confront the known limitations imposed by relativization and the other major lower-bound barriers.*

Proof. Items (a)–(c) follow from the verifier characterization and the SAT reconstruction theorem. Item (d) follows from closure of polynomial time under fixed finite composition. Item (e) follows from the finiteness argument. Item (f) follows from the mismatch between incompleteness hypotheses and complexity-class separation. Item (g) follows because witness-space cardinality does not constrain the existence of structural algorithms. Item (h) follows from the oracle, natural-proof, and algebrization results surveyed above. □ □

10.2 Claims Not Established

For clarity, this paper does *not* establish any of the following claims:

1. $P \neq NP$.
2. That human discovery is asymptotically harder than verification.
3. That all possible AI systems require superpolynomial time to solve NP-complete problems.
4. That multiple independent auditors change the underlying complexity class.
5. That Gödel incompleteness rules out $P = NP$.
6. That modern cryptographic security proves $P \neq NP$.
7. That empirical failure to find polynomial-time algorithms is a mathematical lower bound.
8. That an unresolved philosophical question about the trustworthiness of formal systems invalidates ordinary mathematical proof practice.

These are not omissions accidentally left in the text. They are deliberately excluded because their stronger forms would overstate what has been established.

Lifting and the Semantic Extraction Target

11.1 Lifting Laboratory and the Concrete Extraction Target

The next iteration should not merely enlarge the paper. It should attempt to turn one formally specified obstruction into an actual lower-bound theorem. The communication-trace audit shows that a raw transcript resource is insufficient; any continuation must therefore use a semantically nontrivial resource whose lower bound is not capped by the input length and is not merely a renaming of running time.

11.1.1 A concrete lifting laboratory

The preceding autopsy does not justify the claim that lifting theorems will solve P versus NP . It does, however, justify a more precise experiment: construct a fully explicit lifted problem for which the lower bound can be stated without ambiguity, prove the transfer in that restricted setting, and then identify exactly which step fails when one tries to pass from the lifted communication model to general polynomial-time computation.

Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be an outer Boolean function and let

$$g : X \times Y \rightarrow \{0, 1\}$$

be a two-party gadget. Define the block-composed function

$$(f \circ g^n)(x_1, \dots, x_n, y_1, \dots, y_n) = f(g(x_1, y_1), \dots, g(x_n, y_n)).$$

The intended semantic partition is now intrinsic to the composition: Alice holds the x_i coordinates and Bob holds the y_i coordinates. This is different from arbitrarily cutting the original SAT input string in half. Query-to-communication lifting theorems are precisely designed to exploit this structured partition. [8]

For this laboratory, choose the index gadget

$$\text{Ind}_b(u, j) = u_j, \quad u \in \{0, 1\}^b, \quad j \in [b].$$

Its purpose is not to provide an exotic source of hardness. Its purpose is to create a controlled separation between the information available locally to the two parties and the bit of the outer function that must ultimately be determined.

Definition 11.1.1 (Lifted laboratory family). *For an outer function f and gadget g , the lifted family is*

$$F_{f,g} = f \circ g^n.$$

The outer input length is n , while the distributed input length depends on the encoding length of the gadget inputs.

The first question is completely formal: does lower-bound structure of f survive the lift? For suitable gadget regimes, the answer is yes by established lifting theorems. In particular, the literature contains deterministic, randomized, nondeterministic, and other query-to-communication lifting theorems, and recent work continues to improve both gadget flexibility and the lower-bound models to which lifting applies. [8, 9]

Imported Theorem 11.1.2 (Sensitivity-to-communication lifting for arbitrary gadgets). *Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ have sensitivity s and let $g : X \times Y \rightarrow \{0, 1\}$. Writing $rk(g)$ for the rank of the communication matrix of g , there is a lower bound of the form*

$$D(f \circ g^n) \geq s \left(\frac{\Omega(D(g))}{\log rk(g)} - \log rk(g) \right),$$

with the constants and asymptotic regime specified in the theorem of Iyer.

The imported theorem is not being used as a hidden proof of $P \neq NP$. Its role is diagnostic. It demonstrates that the proposed architecture

outer combinatorial obstruction $\longrightarrow$ gadget lift $\longrightarrow$ communication lower bound

is mathematically real. It also identifies the price of the transfer: the size and structure of the gadget enter the bound, and the resulting communication problem is a deliberately constructed distributed function rather than an arbitrary SAT solver. [8]

11.1.2 Why the lifting laboratory still does not reach $P \neq NP$

The key negative result is as important as the positive theorem.

The lifted function $F_{f,g}$ is a new computational object. A lower bound on $D(F_{f,g})$ does not automatically imply a lower bound on the sequential time needed to solve SAT. A polynomial-time sequential algorithm is not, merely by existing, a two-party protocol for the chosen lifted representation. To obtain such an implication one needs a theorem of the form

arbitrary polynomial-time SAT solver $\longrightarrow$ protocol for $F_{f,g}$

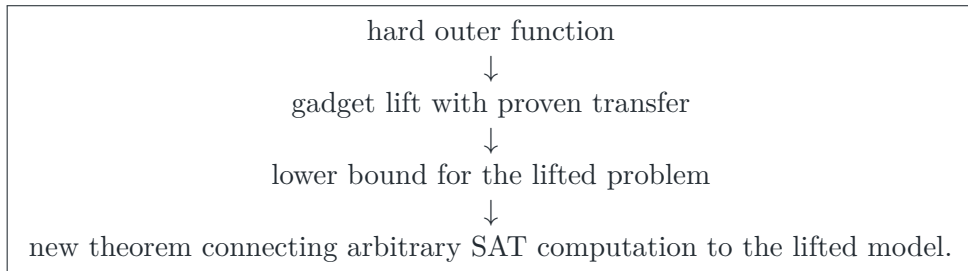
with quantitatively controlled communication.

That transfer is far stronger than the ordinary simulation

$$T_A(x) \longrightarrow O(T_A(x))$$

for a fixed input partition, because the latter only upper-bounds communication by runtime and does not force large communication in the first place.

This gives a necessary three-layer decomposition:



The first three arrows are realizable in substantial restricted settings. The final arrow is the unresolved step relevant to P versus NP .

Proposition 11.1.3 (Lifting is a transfer theorem, not a universality theorem). *A lifting theorem can transfer a lower bound between the specified complexity measures of a composed function. It does not, by itself, imply that every algorithm for an unrelated NP-complete language must instantiate the lifted communication model.*

Proof. The hypotheses of a lifting theorem concern a particular composition $f \circ g^n$ and particular complexity measures. A general SAT algorithm receives a SAT instance under the ordinary encoding and is not required to expose an execution in the lifted protocol class. Therefore an additional reduction from arbitrary SAT algorithms to the lifted model is logically necessary before the communication lower bound can be used as a contradiction to polynomial-time solvability. $\square$ $\square$

11.1.3 A strengthened extraction target

The lifting analysis suggests a more precise extraction map than the raw computation trace. The object extracted from a solver should not be the entire tableau. Instead it should be a *semantic simulation certificate*: a compact object describing how the solver’s computation induces a protocol, circuit, proof, pebbling strategy, or other resource object whose complexity is independently lower-bounded.

Write

$$E_{\text{sem}}(A, x)$$

for such an object, with a resource functional r_{sem} . The target transfer inequality is

$$r_{\text{sem}}(E_{\text{sem}}(A, x)) \leq p(T_A(x), |x|)$$

for a fixed polynomial p , while the desired lower-bound theorem would assert

$$r_{\text{sem}}(E_{\text{sem}}(A, x)) \geq h(|x|)$$

on adversarial inputs, with h superpolynomial.

The phrase “semantic” is a requirement, not an implementation. Any proposed construction is rejected unless it is invariant, or controlled up to polynomial distortion, under innocuous changes of machine description, dead-code insertion, state renaming, and polynomial-time equivalent preprocessing.

This is therefore the concrete extraction target:

construct E_{sem} and prove both transfer and unavoidable lower bound.

11.1.4 Track A: Restricted computational models

Choose a model $\mathcal{M}$ for which strong lower bounds are accessible: for example, decision trees, bounded-width branching programs, restricted circuit classes, restricted proof systems, or communication models. Define a resource $r_{\mathcal{M}}$ and prove an explicit lower bound

$$r_{\mathcal{M}}(x_n) \geq h(n)$$

for a hard family.

The theorem is meaningful only if the model is clearly delimited. A lower bound for $\mathcal{M}$ must never be silently promoted to a lower bound for all polynomial-time algorithms.

11.1.5 Track B: Transfer toward general computation

The crucial bridge is a simulation or decomposition theorem:

$$\text{every polynomial-time solver} \implies \text{an object in } \mathcal{M}$$

whose resource is controlled polynomially. If such a bridge existed with sufficient generality, the restricted lower bound would transfer to P .

This is the step on which many superficially promising lower-bound programs fail. General algorithms can use representations or algebraic identities not visible in the restricted model.

11.1.6 Track C: Algorithm-to-lower-bound duality

Williams's ACC lower-bound program gives a model example of the desired architecture: design nontrivial algorithms for restricted circuit satisfiability and convert those algorithms into circuit lower bounds. [6] The important lesson is methodological, not that the same theorem applies to SAT directly.

A serious programme question is therefore:

$$\begin{array}{c} \text{What algorithmic speedup, in a carefully chosen restricted model,} \\ \text{would force a lower bound?} \end{array}$$

The point is to make the lower-bound engine constructive rather than relying only on an intuition about search-space size.

11.1.7 Track D: Proof complexity

A second route is to study the length or search complexity of propositional proofs. Restricted proof systems admit substantial lower bounds, and the correspondence between short proofs and efficient reasoning is deeply connected to complexity theory. But a universal resolution or bounded-depth lower bound is not automatically a lower bound for unrestricted polynomial-size proofs.

The strongest target must therefore specify a proof system broad enough to imply the desired complexity separation or provide a theorem transferring the restricted proof lower bound to the relevant general system.

11.1.8 Track E: Structural invariants of witnesses

Here the witness-accessibility idea can be made concrete. Candidate invariants include sensitivity-related quantities, certificate complexity, decision-tree measures, expansion parameters, communication complexity, algebraic rank, or topology of solution complexes.

For every candidate, the development protocol is:

$$\begin{array}{l} \text{definition} \rightarrow \text{basic inequalities} \rightarrow \text{hard explicit family} \\ \rightarrow \text{reduction stability} \rightarrow \text{algorithmic lower-bound transfer} \rightarrow \text{barrier audit.} \end{array}$$

The project should stop or redirect as soon as one arrow fails for a structural reason.

11.1.9 Track F: Adversarial computation

Experiments should be designed primarily to kill conjectures.

For a proposed invariant Γ , construct instances that deliberately vary:

$|W_x|$, preprocessing structure, variable symmetry, representation, solver architecture, while attempting to keep the other quantities fixed. A useful invariant should survive these perturbations or explain precisely why the perturbation changes its value.

The output of an experiment should therefore be one of:

supported conjecture, counterexample, ambiguous measurement,

never “empirical proof of $P \neq NP$.”

11.1.10 Stopping criterion

The mathematical stopping criterion is deliberately finite. We stop calling an argument “promising” and either promote it to a theorem or reject/repair it when:

- (1) all objects are formally defined;
- (2) the asymptotic regime is specified;
- (3) the computational model is explicit;
- (4) the principal implication has been proved;
- (5) major counterexamples have been tested;
- (6) all conditional hypotheses are isolated;
- (7) known barriers have been audited;
- (8) no sociological or epistemological premise is carrying an unstated mathematical quantifier.

This is not an invitation to infinite skepticism. It is a finite checklist for whether the argument has reached a mathematically stable endpoint.

11.1.11 Iterative audit and diminishing returns

The intended audit cycle is

proof attempt $\longrightarrow$ adversarial audit $\longrightarrow$ repair $\longrightarrow$ independent re-audit $\longrightarrow$ revision,

with the cycle repeated as long as substantive defects or new mathematical directions continue to emerge.

The crucial qualification is that the number of iterations is not itself a measure of quality. An audit contributes meaningful evidence only to the extent that it has the capacity to detect a failure that previous audits were unlikely to detect. Ten substantially correlated audits can therefore provide less information than a smaller number of genuinely heterogeneous audits. Likewise, an iteration that changes notation or exposition without altering the logical dependency structure should not be regarded as equivalent to an iteration that discovers and repairs a genuine mathematical defect.

This suggests that an audit programme should distinguish at least four quantities. First, let C_t denote the set of substantive criticisms identified up to iteration t . Second, let R_t denote the set of mathematical repairs introduced by iteration t . Third, let U_t denote the set of unresolved high-severity objections after iteration t . Finally, let G_t denote the effective information gained by the t -th audit cycle, where G_t is not merely the number of comments but the amount by which the space of plausible failure modes, unresolved assumptions, or competing interpretations has been reduced.

The sequence

$$G_1, G_2, G_3, \dots$$

need not be monotone in any formal mathematical sense, and there is no universal theorem asserting that it must converge to zero. Nevertheless, a practical project may encounter a regime in which successive independent audits produce increasingly small changes to the mathematical object:

$$G_{t+1} \ll G_t$$

for many successive cycles. This is the sense in which the project may exhibit diminishing returns from further auditing.

A reasonable stopping rule should therefore not be expressed as “stop after N audits,” because no universal value of N can be justified independently of the complexity, novelty, and structure of the research object. A more defensible rule is conditional:

stop a particular audit cycle when

- (i) the principal mathematical objects are defined,
- (ii) the stated claims have explicit logical status,
- (iii) all identified high-severity objections have been resolved, weakened, or explicitly incorporated as hypotheses or open problems,
- (iv) independent audits continue to produce only low-impact changes, and
- (v) the expected information gain from another audit is judged insufficient relative to its cost.

This is not a criterion of truth. It is a criterion for ending a particular round of quality improvement.

That distinction is especially important for P versus NP . No finite number of successful audits can transform a conjectural separation into a theorem if the universal lower-bound argument has not actually been proved. Likewise, no finite number of failed attempts to prove $P \neq NP$ establishes $P = NP$. The mathematical alternatives remain determined by the underlying theorem, not by the number of reviewers.

The practical question is therefore different. Suppose that a research programme concerning a high-stakes mathematical claim has undergone a large number of non-trivial audits. Suppose those audits are heterogeneous rather than merely repetitive; that they contain independent adversarial attempts to construct counterexamples; that proposed repairs are themselves re-audited; that computational experiments are independently reproducible; that definitions and proof dependencies have been repeatedly checked; that known barriers and failure modes have been explicitly addressed; and that successive iterations yield progressively smaller substantive improvements. At what point is it rational, under a finite resource budget, to make a forced binary decision about the current status of the work?

More precisely, how many non-trivial, appropriately heterogeneous audits, together with how much substantive iterative improvement, are sufficient for a complex high-stakes research programme to justify a bounded pragmatic decision when further auditing exhibits only

marginally diminishing returns? This question is deliberately not presented as a theorem of mathematics. It is a question about epistemic and methodological decision-making under finite resources. It also has no obvious universal numerical answer.

The appropriate stopping threshold may depend on the complexity of the mathematical object, the novelty of the definitions, the consequences of an undetected error, the diversity and independence of the auditing mechanisms, the availability of formal verification, the severity distribution of unresolved objections, and the extent to which the latest audits are capable of discovering failure modes that earlier audits could not detect. Consequently, a useful quality protocol should record not merely the number of audits performed, but also their heterogeneity, the classes of failure they were designed to detect, the substantive changes produced by each iteration, the objections that survived, the objections that were disproved, and the reasons for stopping further work.

In this sense, the objective of iterative auditing is not to create an infinite regress of skepticism. Its objective is to make the remaining uncertainty increasingly explicit and increasingly expensive to reduce. The methodological principle can therefore be stated compactly:

Audit relentlessly until the remaining uncertainty is characterized and further substantive audits exhibit sufficiently low marginal yield.

At that point, a practical binary decision may be unavoidable. But the decision should be reported as a decision under bounded audit and resource constraints, not confused with the mathematical truth of the underlying proposition.

11.1.12 A possible computational experiment

A useful experimental companion would enumerate SAT instances across controlled families and record separately:

verification time, search time, certificate length,
backtracking depth, proof length, structural parameters.

It should compare multiple representations and solvers rather than treating one implementation as the definition of computational difficulty.

Particular adversarial families should include:

- (a) many witnesses but easy search;
- (b) few witnesses but easy search;
- (c) unique witnesses with exploitable algebraic structure;
- (d) many witnesses whose naive search is nevertheless hard;
- (e) unsatisfiable instances with short versus long refutations;
- (f) equivalent instances related by polynomial-time preprocessing;
- (g) instances where a proposed topological or sensitivity statistic changes dramatically while optimal solver complexity appears stable.

The experimental objective is to discover theorems worth proving and counterexamples worth preserving. It is not to manufacture a numerical substitute for the missing asymptotic theorem.

Semantic Residual Extraction

12.1 Why a semantic extraction object is needed

The previous stages of the programme identify a recurring failure mode. A proposed hardness quantity is often attached to a particular machine description, a particular trace, or a particular instance. Such an object can be useful for experiments, but it does not automatically survive the quantifier structure of the P versus NP question. The present chapter therefore changes the level of abstraction.

The central object is not the description of how a solver behaves on one input. It is the *semantic witness-extension function* of the underlying search problem. For SAT, this becomes the semantic residual function: it records, for every partial assignment, whether at least one satisfying completion remains. The object is finite, explicit, and independent of implementation.

An important correction is necessary before proceeding. This extension operation is not introduced here as an unprecedented primitive. Closely related monotone-extension and SAT-representation constructions already appear in the literature. In particular, Gwynne and Kullmann give a canonical monotonisation that encodes partial assignments and connect it to monotone circuit complexity, while Okol’nishnikova studies monotone extension and geometrical projection as complexity-changing operations. [23, 22] The present manuscript therefore adopts a more precise claim: it uses the complete semantic extension profile as the organizing object for a uniform, algorithm-to-circuit lower-bound programme, makes its prior-art relations explicit, and develops the resulting research frontier systematically.

The architecture has three layers:

$$\boxed{\text{semantic extension profile} \longrightarrow \text{uniform realization} \longrightarrow \text{lower-bound engine.}}$$

The first layer is semantic; the second supplies an explicit realization produced from an algorithm; the third is where the unresolved complexity-theoretic difficulty resides. The construction does not assume that the semantic object is hard. That would merely rename the desired theorem.

12.2 Partial assignments and residual satisfiability

Let

$$\mathbb{T} = \{0, 1, \star\},$$

where $\star$ means “unassigned.” For a Boolean formula φ with ordered variables $z_1, \dots, z_n$, a vector

$$\rho = (\rho_1, \dots, \rho_n) \in \mathbb{T}^n$$

is a partial assignment. Write $\rho \preceq \sigma$ when every coordinate fixed by ρ is fixed to the same value by σ ; equivalently, σ extends the information in ρ .

The residual formula $\varphi \upharpoonright \rho$ is obtained by substituting the fixed coordinates and leaving the $\star$ -coordinates free. The exact syntactic simplification procedure is irrelevant to the semantic definition; only its set of satisfying completions matters.

Definition 12.2.1 (Semantic residual function). *For a Boolean formula φ on n ordered variables, define*

$$F_\varphi : \mathbb{T}^n \rightarrow \{0, 1\}$$

by

$$F_\varphi(\rho) = 1 \iff \varphi \upharpoonright \rho \text{ is satisfiable.}$$

Equivalently,

$$F_\varphi(\rho) = 1 \iff \exists a \in \{0, 1\}^n \text{ extending } \rho \text{ such that } \varphi(a) = 1.$$

The function F_φ is the proposed semantic core of E_{sem} . It records not one satisfiability answer, but the entire residual satisfiability profile of the instance.

To speak of ordinary Boolean circuit complexity, fix once and for all the two-bit encoding

$$\eta : \mathbb{T} \rightarrow \{0, 1\}^2, \quad \eta(0) = 00, \quad \eta(1) = 01, \quad \eta(\star) = 10.$$

Encode a partial assignment coordinatewise. Let $N(\varphi) = n$ and let $\widehat{F}_\varphi$ be the Boolean function on $2n$ bits that agrees with F_φ on valid encodings and outputs 0 on invalid coordinate encodings. Because the encoding alphabet is fixed and of constant size, changing to any other fixed constant-size encoding changes circuit size by at most a constant-factor input translation. All polynomial-size versus superpolynomial distinctions below are therefore stable under this choice.

12.2.1 Extendibility and the terminology of partial assignments

The predicate used here is an *extendibility* predicate:

$$F_\varphi(\rho) = 1 \iff \exists a \succeq \rho \varphi(a) = 1.$$

It should not be confused with the distinct notions of partial-assignment verification or entailment studied in the SAT and knowledge-representation literature. In particular, the present object asks whether *some* total completion satisfies the formula, rather than whether the partial assignment already forces the formula to be true. Recent work shows that these notions can behave differently outside restricted formula classes. [18]

The construction is also related to classical restriction and subfunction methods in Boolean function complexity, and to conditioning and transformation operations in knowledge compilation. Those connections are not being claimed as new discoveries. The present contribution is to collect the complete semantic extension profile of a SAT instance and then use that profile as the object of an algorithm-to-circuit characterization. [12, 17]

Novelty boundary. The manuscript does not claim novelty for partial assignments, restriction of Boolean functions, conditioning, existential extension, monotone extension, geometrical projection, or the general idea that a Boolean function can be represented by a circuit family. Those constructions have substantial prior literature. The claim advanced here is narrower: the complete existential extension profile is used as the common semantic interface through which the earlier verification–generation programme is connected to a fixed universal SAT family, P-uniform realization, constructive refutation, proof-complexity resources, and meta-complexity. This distinction is deliberate and should be preserved in every statement of contribution.

The terminology “residual” is retained because it emphasizes the residual formula $\varphi \upharpoonright \rho$, but the semantic property itself is more precisely an *existential extension profile*. For a partial assignment ρ , the profile asks whether the subcube of total assignments extending ρ intersects the satisfying set. This geometric interpretation will be used below to separate semantic facts from representation-dependent properties.

Lemma 12.2.2 (Monotonicity under information). *If $\rho \preceq \sigma$, then*

$$F_\varphi(\sigma) \leq F_\varphi(\rho).$$

Proof. Every completion of σ is also a completion of ρ . Thus, if $\varphi \upharpoonright \sigma$ is satisfiable, the same total assignment witnesses that $\varphi \upharpoonright \rho$ is satisfiable. $\square$ $\square$

Lemma 12.2.3 (Root and leaf semantics). *Let $\star^n$ denote the all-unassigned partial assignment. Then*

$$F_\varphi(\star^n) = 1 \iff \varphi \in \text{SAT}.$$

Moreover, if $\alpha \in \{0, 1\}^n$ is total, then

$$F_\varphi(\alpha) = \varphi(\alpha).$$

Proof. At the root, the completions are all assignments, so the first equivalence is the definition of satisfiability. At a total assignment α , there is exactly one completion, namely α itself. $\square$ $\square$

12.3 Subcube interpretation and canonical extension

Let

$$S_\varphi = \{a \in \{0, 1\}^n : \varphi(a) = 1\}$$

be the satisfying set of φ . Every partial assignment $\rho \in \mathbb{T}^n$ determines a subcube

$$C_\rho = \{a \in \{0, 1\}^n : a \succeq \rho\}.$$

The residual function can then be written as

$$F_\varphi(\rho) = 1 \iff C_\rho \cap S_\varphi \neq \emptyset.$$

Proposition 12.3.1 (Subcube characterization). *For every formula φ and partial assignment ρ ,*

$$F_\varphi(\rho) = \bigvee_{a \succeq \rho} \varphi(a).$$

Consequently, F_φ is the downward closure, in the information order, of the satisfying assignments of φ .

Proof. The right-hand side is one exactly when at least one total assignment extending ρ satisfies φ , which is precisely the defining condition for $F_\varphi(\rho) = 1$. The downward-closure statement follows because if $\rho \preceq \sigma$ and $F_\varphi(\sigma) = 1$, any satisfying completion of σ is also a completion of ρ . $\square$ $\square$

This proposition is elementary, but it is conceptually important. It shows that the complete profile is not an additional source of semantic information about the Boolean function: it is a canonical closure operation applied to that function. The research question therefore cannot be “does the profile contain more information than φ ?” It does not. The meaningful question is instead whether passing from a compact representation of φ to a compact representation of its complete extension profile necessarily incurs a large representation cost.

That is exactly the phenomenon studied, in different languages and computational models, by work on monotone extension, geometrical projection, and SAT representations. [22, 23]

12.3.1 A monotone encoding of the information order

The two-bit encoding used above is convenient for invalid-input handling, but it hides a useful structural fact. Define instead

$$\eta_{\text{mon}}(0) = 10, \quad \eta_{\text{mon}}(1) = 01, \quad \eta_{\text{mon}}(\star) = 11,$$

with 00 reserved for an invalid coordinate. Under the ordinary bitwise order, revealing less information corresponds to moving from either 10 or 01 to 11.

Proposition 12.3.2 (Monotone-extension representation). *Let $\hat{F}_\varphi^{\text{mon}}$ be the Boolean function obtained from F_φ using η_{mon} , with invalid encodings mapped to 0. Then $\hat{F}_\varphi^{\text{mon}}$ is monotone. Moreover, the original encoding $\hat{F}_\varphi$ and $\hat{F}_\varphi^{\text{mon}}$ simulate one another by fixed constant-size input translations. Consequently, their circuit complexities agree up to a constant factor.*

Proof. On valid encodings, the only information-order extensions of a coordinate are $10 \preceq 11$ and $01 \preceq 11$. Thus moving upward in the bitwise order can only remove information. By the monotonicity lemma, removing information cannot change a one-value to a zero-value. Invalid 00 inputs are mapped to zero, so they do not break global monotonicity.

There are only finitely many coordinate states. Hence a fixed Boolean circuit can translate either encoding into the other on all four possible two-bit inputs. Applying these translations independently to all coordinates gives linear-size preprocessing and therefore only a constant-factor circuit-size change. $\square$ $\square$

The proposition places the present semantic object directly inside the classical monotone-extension viewpoint. Gwynne and Kullmann explicitly study a monotonisation of a Boolean function in which variables are duplicated so that partial assignments are represented in a monotone Boolean function, and they relate polynomial-size representations of such objects to monotone circuits. [23] Okol’nishnikova’s earlier work similarly studies monotone extension and geometrical projection as operations capable of increasing complexity in several circuit-like models. [22]

The consequence for the present programme is methodological rather than cosmetic: any claim that the raw residual operation itself constitutes a new lower-bound primitive would ignore known prior art. The appropriate novelty question is whether the universal uniform family and the subsequent lower-bound architecture expose a useful new target.

Imported Theorem 12.3.3 (Monotonisation and monotone-circuit characterization). *Gwynne and Kullmann prove, in their SAT-representation framework, an equivalence between polysize CNF representations satisfying their universal-refutation condition and polynomial-size monotone circuits for the corresponding monotonised Boolean functions. In particular, for a sequence of Boolean functions (f_n) , their Theorem 6.5 identifies polysize UR-representations with polynomial-size monotone circuits for the monotonisations $\hat{f}_n$. [23]*

This imported result is close enough to the present semantic construction that it changes how the novelty claim must be phrased. The present volume does not rediscover monotone extension. Instead, it asks whether the universal extension family can serve as a common semantic target across several lower-bound formalisms and whether that target can be attacked beyond monotone or representation-specific complexity.

12.3.2 Extension complexity as a representation gap

For a Boolean function f , let $MC(f)$ denote general Boolean circuit complexity and let $MExt(f)$ denote the minimum size of a monotone circuit computing the canonical extension $\hat{f}^{\text{mon}}$. These are different measures of two related objects. There is no general reason for $MExt(f)$ to be polynomially related to $MC(f)$.

This representation gap is already present in the literature. Monotone extensions can become substantially more complicated than the original functions in several representation models, and SAT-representation work uses precisely this phenomenon to relate propagation quality to monotone circuit complexity. [22, 23]

Important limitation. A large value of $MExt(f)$ is a lower bound in a monotone computational model. It is not, by itself, a lower bound on $MC(f)$, on deterministic time, or on P versus NP . The modern matching literature gives a vivid calibration: the perfect matching function has a $2^{n^{\Omega(1)}}$ monotone-circuit lower bound, while the ordinary decision problem remains polynomial-time solvable. [26] Thus monotone-extension hardness must be treated as a representation phenomenon until an independent transfer theorem to general computation is supplied.

12.4 What the prior literature changes about the research claim

The literature audit changes the status of the semantic object in a precise way.

1. The existence of an existential completion profile is established prior art.
2. Its monotonicity under an appropriate encoding is established prior art.
3. Connections between partial-assignment reasoning and SAT-representation quality are already well developed.
4. What remains legitimate to study here is the use of a fixed universal family of complete extension profiles as a common target for uniform circuit realization and lower-bound transfer.
5. Any future claim of novelty must therefore attach to a theorem about that packaged target, a new reduction, a new lower-bound invariant, a new refutation procedure, or a new transfer theorem—not merely to the definition of F_φ .

This boundary makes the programme stronger, not weaker. It prevents a known construction from carrying an inflated novelty claim while leaving open the more substantive question: whether complete extension profiles reveal a tractable route to an unrestricted lower bound.

12.5 The semantic extraction object

A function alone is not enough for the intended lower-bound transfer because the argument needs to record how a concrete solver realizes that function.

Fix a standard binary encoding of Boolean formulas, together with a canonical polynomial-time transformation

$$\text{enc}(\varphi, \rho)$$

that produces an encoding for the residual instance. Any two reasonable encodings may differ by polynomial-time translation; the representation audit below makes this dependence explicit.

Definition 12.5.1 (Algorithm-relative semantic extraction). *Let A be a deterministic algorithm that correctly decides SAT on the chosen encoding. For a formula φ , define*

$$E_{\text{sem}}(A, \varphi) = (F_\varphi, C_{A, \varphi}),$$

where F_φ is the semantic residual function and $C_{A, \varphi}$ is a Boolean circuit that computes the fixed Boolean encoding $\widehat{F}_\varphi$ by evaluating A on the canonical residual encoding.

The pair intentionally separates semantic and syntactic information. The first component depends only on the Boolean semantics of the instance and the ordering of its variables. The second component is a witness that a particular solver can realize that semantics efficiently.

This is the concrete object requested by the programme: it is finite, algorithm-relative without being syntactically identical to the algorithm, and its semantic component can be compared across different implementations once the variable ordering and Boolean encoding are fixed.

12.5.1 Witness-extension generalization

The SAT residual is one instance of a more general semantic operator. Let $R(x, w)$ be a polynomially bounded binary relation equipped with a prefix encoding as in Definition 2.2.1. Define

$$\text{Ext}_R(x, u) = 1 \iff \exists w \succeq u \ R(x, w) = 1.$$

Thus $\text{Ext}_R(x, u)$ asks whether the current partial witness u can still be extended to a valid witness. For SAT, R is formula satisfaction, u is a partial assignment, and Ext_R is exactly the residual function above.

For inputs of length m , pad the pair (x, u) to a fixed polynomial-size Boolean encoding and let E_m denote the resulting Boolean function. The following theorem shows that the semantic extension family is not an ad hoc SAT construction.

Theorem 12.5.2 (Uniform witness-extension characterization). *Let R be a polynomially balanced, polynomial-time decidable relation admitting the prefix-extension procedure of Definition 2.2.1, and let $L_R = \{x : \exists w \ R(x, w) = 1\}$. Then*

$$L_R \in \mathbf{P} \iff (E_m)_{m \geq 1} \text{ has } P\text{-uniform polynomial-size circuits.}$$

Proof. If $L_R \in \mathbf{P}$, the prefix-extension procedure reduces each query $\text{Ext}_R(x, u)$ to an instance of L_R of polynomial size. Hence the entire Boolean function E_m is computable in polynomial time and admits a P-uniform polynomial-size circuit family by the standard computation-tableau construction.

Conversely, if a P-uniform polynomial-size circuit family computes E_m , evaluate the circuit on the empty prefix $u = \epsilon$. By definition, $E_m(x, \epsilon) = 1$ exactly when $x \in L_R$. This decides L_R in polynomial time. $\square$

This theorem identifies the broader semantic object. The SAT residual family is the canonical NP-complete specialization; the lower-bound question is whether its complete extension profile admits efficient uniform realization.

12.6 Time-to-circuit transfer for the extracted residual function

Lemma 12.6.1 (Residual encoding has polynomial overhead). *There is a polynomial q such that, for every formula φ with n variables and every $\rho \in \mathbb{T}^n$,*

$$|\text{enc}(\varphi, \rho)| \leq q(|\varphi| + n).$$

Proof. A partial assignment uses a constant-size alphabet per variable, and the formula is copied once. Canonical simplification may either be performed explicitly or encoded by a fixed residual interpreter. Either choice gives polynomial, indeed essentially linear, encoding overhead. $\square$ $\square$

Theorem 12.6.2 (Semantic residual circuit extraction). *Suppose A is a deterministic SAT decider with worst-case running time bounded by $t(m)$ on inputs of length m . Then, for every formula φ , there is a Boolean circuit $C_{A,\varphi}$ computing $\hat{F}_\varphi$; equivalently,*

$$C_{A,\varphi}(\eta(\rho)) = F_\varphi(\rho) \quad \text{for every valid } \rho \in \mathbb{T}^n,$$

and

$$|C_{A,\varphi}| \leq \text{poly}(t(q(|\varphi| + n)))$$

for a fixed polynomial independent of A and φ . In particular, if A runs in polynomial time, then $C_{A,\varphi}$ has size polynomial in $|\varphi| + n$.

Proof. On input ρ , form the canonical encoding of the residual instance and run A . By correctness of A , the output is exactly $F_\varphi(\rho)$. A deterministic computation of at most T steps can be unrolled into a Boolean circuit of size polynomial in T by the standard computation-tableau construction. Taking

$$T = t(q(|\varphi| + n))$$

gives the claimed circuit. $\square$

The theorem is deliberately only the algorithmic half of the construction. It does not say that every polynomial-size circuit for one fixed F_φ gives a uniform SAT algorithm. The uniform universal family is needed for that.

12.7 Why pointwise hard-coding is no longer enough

The earlier hard-coding obstruction remains completely correct. For a fixed input, a correct solver can hard-code the answer. A semantic residual object changes the scope of what must be represented.

For a formula with n variables, the residual function has

$$3^n$$

possible partial assignments. A hard-coded answer for the single root $\star^n$ fixes only one value of F_φ . Hard-coding a finite collection of residual answers fixes only finitely many points of the profile.

This is *not* an exponential circuit lower bound: a function on 3^n points may still have a compact circuit. The valid conclusion is narrower:

Hard-coding audit. Pointwise hard-coding destroys instance-only lower bounds, but it does not automatically destroy a lower bound for an entire semantic residual function. The new object therefore survives the specific quantifier attack that invalidated the original universal $\Gamma(x)$ proposal.

12.8 A sharp counterexample: some residual profiles are trivial

Proposition 12.8.1 (Unsatisfiable and tautological profiles). *If φ is unsatisfiable, then*

$$F_\varphi \equiv 0.$$

If φ is a tautology, then

$$F_\varphi \equiv 1.$$

Proof. An unsatisfiable formula has no satisfying completion under any partial assignment; a tautology remains satisfiable under every partial assignment. □ □

Thus no theorem of the form

$$F_\varphi \text{ is hard for every } \varphi$$

can hold. Any hardness theorem must concern an explicit family that contains arbitrary SAT instances as semantic restrictions.

12.9 Representation dependence and polynomial stability

Proposition 12.9.1 (Variable-renaming stability). *Let π be a permutation of the variables of φ , and let φ^π be the renamed formula. Then F_{φ^π} is obtained from F_φ by the corresponding permutation of partial-assignment coordinates. Consequently, Boolean circuit size changes only by input rewiring.*

Proof. A partial assignment for the renamed variables corresponds bijectively to one for the original variables, and the corresponding residuals are satisfiable simultaneously. Permuting circuit input wires therefore gives the other representation. □ □

Proposition 12.9.2 (Dummy-variable stability). *Let*

$$\varphi'(z, y) = \varphi(z)$$

where y is fresh. Then

$$F_{\varphi'}(\rho, \eta) = F_{\varphi}(\rho)$$

for all partial assignments (ρ, η) . Hence a circuit for F_{φ} gives one for $F_{\varphi'}$ by ignoring the dummy coordinate.

Proof. The fresh variable imposes no constraint, so the existence of a satisfying completion depends only on ρ . □

Definition 12.9.3 (Polynomially stable residual reduction). *A transformation from a family (φ_n) to a family $(\psi_{m(n)})$ is polynomially stable if there are polynomial-time maps sending relevant partial assignments for φ_n to partial assignments for $\psi_{m(n)}$, preserving residual satisfiability, with $m(n) \leq \text{poly}(n)$.*

Proposition 12.9.4 (Circuit upper bounds transfer under stable residual reductions). *Suppose (φ_n) polynomially reduces to $(\psi_{m(n)})$ through a polynomially stable residual reduction. Then any polynomial-size circuit family for the residual functions of $\psi_{m(n)}$ yields a polynomial-size circuit family for the residual functions of φ_n .*

Proof. Compose the target residual circuit with circuits implementing the polynomial-time partial-assignment map and the output-preserving restriction. Circuit composition preserves polynomial size. □

12.10 The residual satisfiability problem is NP-complete

The residual profile should not be mistaken for a problem outside the ordinary NP-completeness landscape. It is useful to state its decision complexity explicitly, because the lower-bound programme concerns the representation of the *whole* profile rather than the complexity of one extension query in isolation.

Definition 12.10.1 (Residual-SAT). *Define*

$$\text{RES-SAT} = \{(\varphi, \rho) : F_{\varphi}(\rho) = 1\},$$

where the pair (φ, ρ) is encoded in the fixed residual representation used throughout this volume.

Theorem 12.10.2 (Residual-SAT completeness). *RES-SAT is NP-complete under polynomial-time many-one reductions.*

Proof. Membership in NP is immediate. On input (φ, ρ) , a verifier guesses a complete assignment $a \succeq \rho$ and evaluates $\varphi(a)$. The witness has length polynomial in the input size, and the formula evaluation is polynomial-time. Thus $(\varphi, \rho) \in \text{RES-SAT}$ exactly when there exists an accepted witness.

For hardness, map a SAT instance φ with variables $x_1, \dots, x_n$ to $(\varphi, \star^n)$. By definition,

$$F_{\varphi}(\star^n) = 1 \iff \varphi \text{ is satisfiable.}$$

The map is polynomial-time. Hence $\text{SAT} \leq_p \text{RES-SAT}$. Since SAT is NP-complete, RES-SAT is NP-complete. □

The theorem closes an otherwise easy ambiguity. A single residual query is not a new complexity class: it is an NP-complete extension problem. The distinctive object studied here is instead the *complete semantic profile*, namely the requirement to represent all such extension answers simultaneously and uniformly. This is also why the distinction between one query, a family of residual queries, and a compact representation of the complete profile must remain explicit throughout the lower-bound programme.

12.11 A universal semantic family

Fix a polynomial-time uniform family of Boolean formulas

$$U_m(Y, Z),$$

with instance variables Y and auxiliary variables Z , such that

$$y \in \text{SAT} \iff U_m(y, Z) \text{ is satisfiable.}$$

A standard Cook–Levin/Karp reduction can be used to ensure that the family has polynomial size, and it may be taken to be CNF or 3-CNF.

Regard U_m as a formula on all variables (Y, Z) . Let $N(m)$ denote its total number of variables; the construction is uniform and $N(m)$ is polynomial in m . Set

$$F_m := F_{U_m}, \quad \hat{F}_m := \hat{F}_{U_m}.$$

For $y \in \{0, 1\}^m$, let ρ_y assign $Y = y$ and leave every Z -variable unassigned.

Lemma 12.11.1 (Universal restriction identity). *For every $y \in \{0, 1\}^m$,*

$$F_m(\rho_y) = 1 \iff y \in \text{SAT}.$$

Proof. By definition, $F_m(\rho_y) = 1$ exactly when the residual formula $U_m(y, Z)$ is satisfiable. The universal-family property gives the equivalence. $\square$ $\square$

12.12 The uniform circuit equivalence

Definition 12.12.1 (P-uniform polynomial-size residual circuits). *The family (F_m) has P-uniform polynomial-size circuits if there exists a deterministic polynomial-time constructor G and a polynomial p such that $G(1^m)$ outputs a circuit C_m of size at most $p(m)$ computing the fixed Boolean encoding $\hat{F}_m$.*

Theorem 12.12.2 (Uniform semantic-residual characterization). *For the universal residual family (F_m) ,*

$$\text{SAT} \in \text{P} \iff (F_m) \text{ has P-uniform polynomial-size circuits.}$$

Consequently,

$$\text{P} = \text{NP} \iff (F_m) \text{ has P-uniform polynomial-size circuits.}$$

Proof. If $\text{SAT} \in \text{P}$, let A be a polynomial-time SAT decider. The formulas U_m are uniformly constructible and have polynomial size. The semantic residual circuit extraction theorem therefore produces, uniformly in m , polynomial-size circuits computing $\hat{F}_m$. Because A is fixed

and the residual encoding and computation-tableau construction are polynomial-time effective, the circuit constructor $G(1^m)$ can construct these circuits within polynomial time.

Conversely, suppose a polynomial-time circuit constructor outputs C_m of polynomial size. On input y of length m , construct C_m , encode the partial assignment ρ_y , and evaluate C_m at ρ_y . The universal restriction identity shows that the result is the SAT answer. The overall running time is polynomial in m . Hence $\text{SAT} \in \text{P}$. Since SAT is NP-complete, this is equivalent to $\text{P} = \text{NP}$. $\square$ $\square$

12.13 A strong sufficient lower-bound target

Theorem 12.13.1 (Semantic residual circuit lower bound implies $P \neq NP$). *Assume that there is a superpolynomial function h such that*

$$\text{CC}(\hat{F}_m) > h(m)$$

for infinitely many m , where h eventually dominates every polynomial. Then

$$\text{P} \neq \text{NP}.$$

Proof. If $\text{P} = \text{NP}$, then SAT belongs to P. The uniform semantic-residual characterization gives P-uniform polynomial-size circuits for the fixed Boolean encoding $\hat{F}_m$, and hence ordinary polynomial-size circuits. This contradicts the assumed superpolynomial lower bound on infinitely many m . $\square$ $\square$

Proposition 12.13.2 (Nonuniformity warning). *Merely knowing that a semantic family has small nonuniform circuits does not imply $\text{SAT} \in \text{P}$, because the circuit family need not be efficiently constructible. By contrast, a superpolynomial lower bound against all nonuniform circuits would be more than sufficient for $P \neq NP$.*

Proof. A P-uniform family is nonuniform, so a lower bound against all polynomial-size nonuniform circuits rules out the P-uniform case. The converse direction fails because nonuniform circuits may contain arbitrary advice not computable in polynomial time. $\square$ $\square$

12.13.1 The lower-bound hierarchy

The exact logical target is uniform, while several stronger targets are available as convenient sufficient conditions:

$$\begin{aligned} \text{Exact target} & : (F_m) \notin \text{P-uniform-SIZE}[m^{O(1)}], \\ \text{Stronger target} & : \text{CC}(\hat{F}_m) \text{ is superpolynomial,} \\ \text{Very strong target} & : \text{CC}(\hat{F}_m) \geq 2^{m^\epsilon} \text{ for some } \epsilon > 0. \end{aligned}$$

The second and third statements are stronger than necessary. Keeping this distinction explicit prevents the research programme from treating one convenient route to $P \neq NP$ as though it were the only logically available route.

12.14 Is the semantic object merely SAT in disguise?

A legitimate objection is that the residual construction could be another encoding of SAT under a larger domain. The universal restriction identity confirms that the object is not automatically easier than SAT.

The contribution is therefore not a new complexity class. It is a clean separation of three tasks:

$$\begin{aligned} \text{instance semantics} &\rightarrow F_m, \\ \text{solver realization} &\rightarrow C_m, \\ \text{lower-bound engine} &\rightarrow \text{CC}(\widehat{F}_m). \end{aligned}$$

A successful proof must attack the third line without defining it in terms of the unknown answer to the first two.

12.15 Residual subfunction profiles and classical lower-bound connections

The semantic residual function also admits a block-wise profile closely related to the classical subfunction viewpoint behind Nechiporuk-style lower bounds. Let V be a subset of the formula variables. For each assignment of all variables outside V , the restriction of F_φ to the coordinates in V is a function $\mathbb{T}^V \rightarrow \{0, 1\}$. Define

$$\text{RSub}_V(\varphi) = \{F_\varphi \upharpoonright \rho : \rho \text{ fixes all variables outside } V\}$$

and, for a partition $\Pi = (V_1, \dots, V_k)$ of the variables,

$$\text{RSP}_\Pi(\varphi) = \sum_{i=1}^k \log_2(1 + |\text{RSub}_{V_i}(\varphi)|).$$

The $+1$ is only a harmless convention ensuring that the logarithm is defined when a block has a single residual behavior.

Classical Nechiporuk arguments count Boolean subfunctions induced by restrictions to variable blocks and convert sufficiently many distinct subfunctions into formula-size or related lower bounds. [12] The residual profile above is a semantic analogue for the extension predicate: it counts how many distinct continuation problems can remain on a block once the rest of the instance has been fixed.

Proposition 12.15.1 (Permutation invariance of the residual profile). *If π is a permutation of the variables and $\pi\Pi$ is the correspondingly permuted partition, then*

$$\text{RSP}_{\pi\Pi}(\varphi^\pi) = \text{RSP}_\Pi(\varphi).$$

Proof. Variable renaming induces a bijection between partial assignments and between the corresponding residual subfunctions. Hence each block has exactly the same number of semantic residual behaviors after the corresponding permutation. $\square$ $\square$

This does not by itself provide an unrestricted circuit lower bound. Its purpose is narrower and more concrete: it supplies a classical lower-bound laboratory in which the semantic residual object can be tested against a well-developed subfunction methodology. The key research question is whether a residual-profile lower bound can be obtained in a model broad enough to say more than a restricted formula or branching-program theorem.

A useful audit rule follows immediately: if a proposed lower bound for RSP_Π changes substantially under polynomially equivalent encodings for reasons unrelated to the underlying extension problem, then the statistic is measuring representation rather than semantics and should not be promoted.

12.16 Restricted invariants: what remains useful

The sensitivity and decision-tree discussion remains valuable as a laboratory. Quantities such as

$$\text{DT}(F_m), \quad \text{bsens}(F_m), \quad \text{Peb}(F_m), \quad \text{Proof}(F_m)$$

can be mathematically meaningful.

But each measure comes with its own computational model. A lower bound for decision trees, monotone circuits, bounded-depth circuits, resolution, or communication is not a lower bound against arbitrary polynomial-time algorithms unless an additional transfer theorem is proved.

This is a productive limitation rather than a defect. A restricted lower bound can identify structure worth trying to lift into a general lower bound.

12.17 Proof complexity direction

For ρ with $F_\varphi(\rho) = 0$, the statement that the residual is unsatisfiable can be represented by a propositional refutation. Thus the residual profile can be decorated by proof lengths or proof-system resources.

Research Direction. Search for a proof system in which the universal residual family has a lower-bound certificate that can be transferred, with polynomial overhead, to arbitrary polynomial-time SAT computation.

This remains a research direction. Lower bounds for a restricted proof system are not silently promoted to lower bounds for unrestricted proofs.

12.18 Meta-complexity direction

One may ask whether the residual functions F_m possess small representations in a chosen model, and whether a finite certificate of non-smallness can be checked efficiently. This points toward meta-complexity.

The circularity test is essential. A certificate whose meaning is simply “no small circuit exists” is not yet a constructive lower-bound method. The open task is to find an independently checkable structural witness implying that conclusion.

12.19 Semantic extraction and lifting

The lifting laboratory can now be attached to an explicitly defined semantic family. For a gadget g , consider

$$F_m \circ g^N.$$

Recent lifting theorems give substantial communication lower bounds for carefully chosen compositions; Iyer proves a sensitivity-to-communication lifting theorem for arbitrary gadgets, and later work studies deterministic one-way number-on-forehead lifting. [8, 9]

The attraction of this route is that the outer function is now tied to the NP-complete semantic family. The unresolved issue is still the transfer from a centralized polynomial-time algorithm to a model with a superpolynomial lower bound.

Proposition 12.19.1 (Communication ceiling). *For a distributed function whose total input length is N , deterministic communication complexity is at most $N + 1$. Therefore ordinary communication complexity of the lifted representation cannot alone yield a superpolynomial lower bound in the centralized input length.*

Proof. One party can transmit its entire input to the other, which can then evaluate the function and send one output bit. □ □

A successful lifting-based separation would consequently need another resource, another normalization, or a theorem that couples the semantic representation to a larger computation in a way not capped by the total input length.

12.20 Barrier audit for semantic extraction

12.20.1 Relativization

Baker, Gill, and Solovay constructed recursive oracles under which $P^O = NP^O$ and others under which $P^O \neq NP^O$. [3] A proof of a semantic circuit lower bound must therefore use genuinely unrelativized structure if it is to settle the unrelativized problem by a route that otherwise relativizes.

12.20.2 Natural proofs

Razborov and Rudich identify a barrier for broad classes of constructive, large properties that would yield strong general circuit lower bounds under standard hardness assumptions. [4] The semantic status of F_m does not remove this constraint. A prospective lower-bound property must be tested for the natural-proofs conditions rather than declared immune because it is semantic.

12.20.3 Algebrization

Aaronson and Wigderson identify algebrization as an additional barrier beyond ordinary relativization. [5] A proof using arithmetic extensions or low-degree representations must therefore be audited for the corresponding algebrizing behavior.

12.21 Finite audit protocol for the new object

The semantic family should be subjected to finite transformations before any asymptotic claim is trusted:

1. variable permutations;
2. dummy-variable padding;
3. clause reordering;
4. insertion of syntactically redundant tautologies;
5. polynomial-time preprocessing;
6. equivalent SAT-complete encodings;

7. solver descriptions with arbitrary dead code;
8. different constructions of the same universal family.

The first two have exact stability theorems above. For the others, polynomial distortion is the correct target.

12.22 Classification of the new result

Status: unconditional construction with an open lower-bound target.

The semantic residual function is rigorously defined. A polynomial-time SAT solver induces polynomial-size uniformly constructible circuits for the universal residual family, and conversely such a P-uniform family yields a polynomial-time SAT solver. Therefore a superpolynomial lower bound for the semantic family would prove $P \neq NP$.

No such lower bound is proved here.

This is the strongest unconditional characterization established in the present programme. The manuscript does not claim that the extension operation itself is new; its prior-art status is documented above. The unresolved lower-bound theorem remains the central mathematical frontier.

Structural Decomposition of the Extension Profile

13.1 Free-variable layers

The extension profile lives on 3^n valid partial assignments. It is therefore useful to organize it by the number of unassigned variables. For $\rho \in \mathbb{T}^n$, write

$$\text{free}(\rho) = |\{i : \rho_i = \star\}|.$$

For $0 \leq k \leq n$, define the k -th residual layer

$$L_{n,k} = \{\rho \in \mathbb{T}^n : \text{free}(\rho) = k\}.$$

A direct count gives

$$|L_{n,k}| = \binom{n}{k} 2^{n-k}.$$

The layers have two dual interpretations. When k is small, the current partial assignment leaves only a small number of total completions. When $n - k$ is small, there are only a small number of fixed-variable patterns. Both regimes admit elementary finite representations.

Lemma 13.1.1 (Small-free-variable realization). *Let φ be represented by a Boolean circuit of size s on n variables. For every k , the restriction of F_φ to $L_{n,k}$ can be computed with circuit size*

$$\text{poly}(n, s) 2^k.$$

In particular, if $k = O(\log n)$ and $s = \text{poly}(n)$, the k -th layer has polynomial-size circuits.

Proof. Given a valid partial assignment with exactly k free coordinates, enumerate its 2^k total completions. For each completion, evaluate the circuit for φ , then OR the resulting 2^k bits. A circuit can first verify or decode the coordinate states and then implement this bounded enumeration. The overhead for selecting the free coordinates, routing their values, evaluating φ , and forming the final OR is polynomial in n and s , multiplied by the number of enumerated completions. $\square$ $\square$

Proposition 13.1.2 (Nonuniform constant-fixed-variable boundary). *If $n - k = O(1)$, then the layer $L_{n,k}$ has polynomial-size circuits whenever φ has polynomial-size representation.*

Proof. If $n - k = c$ for a fixed constant c , then

$$|L_{n,k}| = \binom{n}{c} 2^c = n^{O(1)}.$$

A lookup construction can therefore hard-wire the value of F_φ on every valid point of the layer. The resulting circuit has polynomial size. This construction is explicitly nonuniform: producing the hard-wired table need not be polynomial-time effective, and therefore this proposition is not a P-uniform upper bound. $\square$ $\square$

For $n - k$ growing with n , this simple counting argument ceases to be polynomial (for example, $n - k = \Theta(\log n)$ gives only a quasipolynomial bound). We therefore make no stronger boundary-layer claim without an additional structural argument.

Layer audit rule. The small-free-variable theorem and the constant-fixed-variable proposition are genuine polynomial upper bounds. Intermediate values of k require separate analysis. Counting the number of valid partial assignments is not, by itself, a circuit upper bound.

Recent restriction-based work reinforces the model-specific nature of this route. For example, Chalupa gives an explicit AC^0 lower bound for a natural distribution of satisfiable 3-CNFs using random restrictions, with an explicit non-triviality event to exclude degenerate restricted instances. This is a useful calibration for the present layers: restriction methods can preserve hardness in a carefully chosen circuit class, but the proof depends on the class and the restriction distribution and does not transfer automatically to general circuits. [27]

13.2 Semantic mutation and contextual instability

The extension profile also permits a precise version of a useful intuition: a small change in a context can force a large change in the set of completions that remain possible. This is not, by itself, a general-circuit lower bound. It is a structural quantity that can be tested in restricted representation models and used to formulate more precise lower-bound questions.

For a partial assignment ρ , let

$$S(\rho) = \{i : \rho_i = \star\}$$

and define its *completion fibre*

$$W_\varphi(\rho) = \{a \in \{0, 1\}^{S(\rho)} : \varphi(\rho \cup a) = 1\}.$$

Here $\rho \cup a$ denotes the complete assignment obtained by filling the free coordinates of ρ according to a . Thus

$$F_\varphi(\rho) = 1 \iff W_\varphi(\rho) \neq \emptyset.$$

The function F_φ therefore records only the nonemptiness of each completion fibre; the fibre itself contains strictly more semantic information.

Definition 13.2.1 (Contextual mutation). *Let ρ and σ be partial assignments with the same free-variable set $S(\rho) = S(\sigma)$. Their normalized contextual mutation distance is*

$$CMut_\varphi(\rho, \sigma) = \frac{|W_\varphi(\rho) \triangle W_\varphi(\sigma)|}{2^{|S(\rho)|}},$$

where $\triangle$ denotes symmetric difference. A one-bit contextual mutation is a pair for which ρ and σ differ on exactly one fixed coordinate.

The normalization is chosen only to make the quantity lie in $[0, 1]$. It is not claimed to be a canonical complexity measure. Its role is diagnostic: it measures how much of the continuation landscape changes when the current context changes. For a fixed free-variable set, the induced quantity on contexts is in general a pseudometric: distinct contexts may induce exactly the same completion fibre.

Proposition 13.2.2 (A one-bit mutation can cause maximal contextual change). *For every $m \geq 1$, there is a formula φ_m of size $O(1)$ and one-bit contextual mutations ρ, σ with m free variables such that*

$$\text{CMut}_{\varphi_m}(\rho, \sigma) = 1.$$

Proof. Let x be a fixed context bit, let $y_1, \dots, y_m$ be free, and take

$$\varphi_m(x, y) = (x \leftrightarrow y_1).$$

Let ρ fix $x = 0$ and leave all y_i free, while σ fixes $x = 1$ and leaves all y_i free. Then

$$W_\varphi(\rho) = \{a \in \{0, 1\}^m : a_1 = 0\}, \quad W_\varphi(\sigma) = \{a \in \{0, 1\}^m : a_1 = 1\}.$$

These fibres are disjoint and their union is all of $\{0, 1\}^m$. Hence the normalized symmetric difference equals 1. $\square$ $\square$

The proposition is deliberately elementary. Its significance is not the formula itself, but the distinction it exposes: an instance can have a compact syntactic description while the continuation landscape changes abruptly under a tiny change of context. The hard question is whether such contextual changes can be arranged in a way that forces a general computational representation to store many independent contexts.

13.2.1 Context profiles and the memory interpretation

Fix a block B of variables. For a complete assignment α to the variables outside B , define the context profile

$$G_{\alpha, B}(\tau) = F_\varphi(\alpha \cup \tau), \quad \tau \in \mathbb{T}^B.$$

Let

$$\text{Ctx}_B(\varphi) = \left| \left\{ G_{\alpha, B} : \alpha \in \{0, 1\}^{[n] \setminus B} \right\} \right|$$

be the number of distinct continuation profiles induced by complete outside contexts. This is a semantic “instruction-memory” quantity: two contexts can share the same state only if they induce the same answers for every possible partial assignment on B .

The definition is closely related to the classical notion of subfunctions under restrictions. Such restriction complexity is a well-developed subject, and known results show that suitably complicated restrictions can force complexity increases in specific representation models. [15, 12] The present notation packages the same idea around the complete existential-extension profile. It is therefore a research definition, not a claim of a new classical complexity measure.

Proposition 13.2.3 (Context count lower-bounds ordered branching width). *Suppose a deterministic ordered binary decision diagram reads every variable outside B before it reads any variable in B . Then its width immediately after the last outside variable is at least*

$$\text{Ctx}_B(\varphi).$$

Proof. Each assignment α to the outside variables determines the state reached by the ordered decision diagram after those variables have been read. From that state onward, the remaining computation depends only on the variables in B and therefore computes a fixed function of the continuation τ . If two contexts α and β reached the same state, the same remaining subgraph would have to compute

$$G_{\alpha,B} = G_{\beta,B}.$$

Consequently distinct context profiles require distinct states at that cut. The width is therefore at least $\text{Ctx}_B(\varphi)$. $\square$ $\square$

This is a rigorous instance of the “instruction-memory” intuition, but only in a restricted model. Ordered decision diagrams are explicitly built from the idea that a node represents the residual function that remains after a prefix of variables has been fixed; Bryant’s canonical graph representation makes this state/subfunction interpretation explicit. [16]

Example 13.2.4 (Exponential context memory for a small formula). *For*

$$\varphi_m(x, y) = \bigwedge_{i=1}^m (x_i \leftrightarrow y_i)$$

and $B = \{y_1, \dots, y_m\}$, every assignment $\alpha \in \{0, 1\}^m$ to the x -variables induces a different context profile:

$$G_{\alpha,B}(\tau) = 1 \iff \tau \text{ is consistent with } \alpha.$$

If $\alpha \neq \beta$, choose an index j on which they differ and take the partial assignment τ fixing $y_j = \alpha_j$ while leaving the other y -variables free. Then $G_{\alpha,B}(\tau) = 1$ and $G_{\beta,B}(\tau) = 0$. Hence

$$\text{Ctx}_B(\varphi_m) = 2^m.$$

Therefore every ordered decision diagram that reads all x -variables before the y -variables has width at least 2^m at that cut, despite the defining formula having size $O(m)$.

Context-instability audit. A large contextual-mutation count or a large context profile count is not, by itself, a lower bound for unrestricted Boolean circuits. A general circuit need not expose a single read-once state at a fixed cut, and it may share information between many contexts through arbitrary gates. The ordered-branching result above is therefore a calibration theorem: it shows exactly how the intuition becomes a true lower-bound certificate in one restricted model and identifies the missing transfer theorem needed for general circuits.

13.2.2 Why the context route is still worth pursuing

The context profile suggests a more demanding general question than raw sensitivity: can one identify a family for which many contexts induce semantically incompatible continuation profiles, while every small general circuit is forced to reuse some computational resource across those contexts? The desired conclusion would have the form

many semantically distinct contexts $\implies$ forced resource sharing $\implies$ a circuit lower bound.

The first implication can already be made exact in ordered branching models. The second implication is the open part. It is precisely where a constructive gate-elimination argument, a proof-complexity transfer, or another unrestricted-circuit method would have to enter.

This framing also prevents an easy but invalid counting argument. There may be exponentially many contexts even for a function with a tiny circuit. What matters is not the number of contexts alone, but the number of *distinct residual behaviours* and whether those behaviours can be shared by a circuit without loss.

13.3 The residual obstruction profile

Define the complementary obstruction profile

$$U_\varphi(\rho) = 1 - F_\varphi(\rho).$$

Then

$$U_\varphi(\rho) = 1 \iff \varphi \upharpoonright \rho \text{ is unsatisfiable.}$$

Lemma 13.3.1 (Upward closure of obstructions). *If $\rho \preceq \sigma$ and $U_\varphi(\rho) = 1$, then $U_\varphi(\sigma) = 1$.*

Proof. If the weaker partial assignment ρ already has no satisfying completion, then a stronger assignment σ cannot create a satisfying completion. $\square$ $\square$

Thus F_φ is a down-set of extendible partial assignments, while U_φ is the corresponding up-set of obstructed partial assignments.

This duality is useful for lower-bound work because an unsatisfiable residual admits a second semantic decoration: a refutation.

13.4 Residual refutation profiles

Fix a sound complete propositional proof system $\mathcal{P}$. Define

$$\text{Proof}_{\mathcal{P}}(\varphi, \rho) = \begin{cases} \min\{|\pi| : \pi \text{ is a } \mathcal{P}\text{-refutation of } \varphi \upharpoonright \rho\}, & U_\varphi(\rho) = 1, \\ \infty, & F_\varphi(\rho) = 1. \end{cases}$$

This creates a two-coordinate semantic object:

$$(F_\varphi(\rho), \text{Proof}_{\mathcal{P}}(\varphi, \rho)).$$

The first coordinate asks whether the residual is extendible; the second asks how expensively a chosen proof system can certify its obstruction.

The distinction is important. A lower bound on $\text{Proof}_{\mathcal{P}}$ is meaningful only inside the chosen proof system. It becomes a route toward $P \neq NP$ only if one can prove an additional transfer theorem showing that every hypothetical polynomial-time SAT procedure would induce a proof object whose resource is polynomially bounded.

This gives a disciplined version of the proof-complexity route rather than an informal claim that “hard residuals imply hard algorithms.”

Recent work on the Proof Analysis Problem makes the constructive side of this distinction more concrete. Arteche, Atserias, de Rezende, and Khaniki prove that short Resolution proofs of certain formulas stating Resolution lower bounds can be algorithmically mined for satisfying assignments, while the analogous analysis problem for Extended Frege is NP-complete. The result is useful here because it demonstrates a pattern close to the present programme: a proof

object can contain extractable semantic information, but the strength of that extraction depends sharply on the proof system. [19]

The residual question is therefore sharper than “are residual refutations long?” It is: can a proof of residual obstruction be required to reveal a semantic context that a small realization of the extension profile cannot otherwise encode? No general theorem of this kind is claimed.

13.5 Where the lower-bound target can and cannot hide

The layer decomposition yields a useful negative conclusion. Any lower-bound programme that derives all of its force solely from partial assignments with at most $O(\log n)$ free variables cannot prove a superpolynomial lower bound for the complete semantic family, because those layers already admit the elementary 2^k -completion construction.

The remaining possibility is therefore a genuinely global obstruction, or an obstruction arising in layers with a growing number of free variables. This does not prove that middle layers are hard. It only eliminates one overly local proof strategy.

The distinction matters because a full circuit can couple different layers simultaneously. A lower bound for one layer is therefore not automatically a lower bound for the full function, and conversely a full-function lower bound might arise from interactions that do not have a simple layer-by-layer interpretation.

13.6 A structural research question

The preceding observations suggest the following question.

Question 13.6.1 (Intermediate-layer extension problem). *Does there exist an explicit family of formulas (φ_n) such that every polynomial-size general Boolean circuit attempting to compute the complete extension profile of φ_n must fail on a partial assignment having*

$$n^\alpha \leq \text{free}(\rho) \leq n - n^\alpha$$

for some fixed $\alpha > 0$?

This is deliberately weaker than a P versus NP theorem. It is a structural question about where a potential representation obstruction could live. It is also compatible with the known warning that monotone or restricted-model lower bounds do not automatically transfer to general computation.

Constructive Refutation as a Lower-Bound Engine

14.1 Why constructiveness changes the target

A nonconstructive circuit lower bound says that every sufficiently small circuit must fail somewhere. A constructive lower bound additionally supplies an efficient procedure that, given the circuit, finds a failing input. The latter notion has become especially important in modern complexity theory. Chen, Jin, Santhanam, and Williams show that constructiveness is a dividing line between many weak lower bounds and the strong separations sought in complexity theory. [24]

Recent work by Carmosino, Dang, and Jackman makes the connection to gate elimination particularly direct: they give constructive lower bounds using convergent simplification and, in a subsequent preprint, explicit refuters extracted from gate-elimination arguments. [20, 21]

The semantic extension programme should therefore not ask only “is F_m hard?” It should also ask:

can a purportedly small realization of F_m be efficiently refuted?

14.2 Definition of a residual refuter

Definition 14.2.1 (Residual refuter). *Let $G_m : \{0, 1\}^{N_m} \rightarrow \{0, 1\}$ be a Boolean family and let $\mathcal{C}_s$ be a circuit class with size bound $s(m)$. A residual refuter for G at size bound s is a deterministic algorithm $\mathcal{R}$ such that, on input a circuit $C \in \mathcal{C}_s$, the algorithm outputs an input $z \in \{0, 1\}^{N_m}$ satisfying*

$$C(z) \neq G_m(z).$$

For the semantic family, we additionally require that z decode to a valid partial assignment. This avoids allowing a refuter to win merely on syntactically invalid encodings.

Proposition 14.2.2 (Refuter implies separation from the target circuit class). *Suppose a polynomial-time residual refuter exists for G_m against every circuit of size at most m^c , for each fixed constant c . Then G is not in $P\text{-uniform-SIZE}[m^{O(1)}]$.*

Proof. Assume toward contradiction that G has a P-uniform circuit family of polynomial size. Fix its polynomial exponent c . A polynomial-time constructor produces the corresponding circuit C_m , and the refuter produces a valid input on which C_m is wrong. This contradicts the assumption that C_m computes G_m . □

The proposition is elementary, but it gives the programme a constructive target whose quantifiers are correct. A refuter is not itself a lower-bound proof unless its correctness has been established; however, the proof obligation is now explicit.

14.3 Three levels of residual constructivity

It is useful to distinguish three targets.

1. **Fixed-threshold refutation.** For a particular size bound $s(m)$, efficiently find an error in every circuit of that size.
2. **Polynomial-uniform refutation.** Provide a single efficient refuter that works for all polynomial size thresholds, with the threshold supplied as part of the input.
3. **Barrier-resistant refutation.** Give a refuter whose correctness follows from a structural theorem that survives representation changes and does not collapse into a known restricted lower-bound barrier.

The third target is the one that could plausibly interact with the original algorithm-relative resource-transfer programme. The first two are valuable calibration targets even if they do not imply $P \neq NP$.

14.4 Gate elimination as a concrete laboratory

Gate elimination is currently the principal known technique for proving explicit lower bounds against unrestricted Boolean circuits. Li and Yang obtained a $3.1n - o(n)$ lower bound for explicit affine dispersers, and the 2026 work of Carmosino, Dang, and Jackman gives convergent and constructive formulations of gate elimination for several bases and classical lower bounds. [25, 20, 21]

This literature motivates a concrete residual experiment:

$$C \longrightarrow C^*$$

where C^* is a canonical or convergent simplification of a purported circuit for $\widehat{F}_m$. One asks whether a semantic property of the residual family must survive every such simplification and whether every sufficiently small circuit is driven into a normal form that violates that property.

No such theorem is claimed here. The point is that the route is now experimentally and mathematically specific: a proposed invariant can be tested against an actual simplification system, and a successful proof would yield a refuter rather than merely an existential counting argument.

14.5 A contextual-interaction calibration

The contextual-memory intuition can be stress-tested with a toy interaction family before being connected to a lower-bound theorem. Let a context be $c \in \{0, 1\}^m$ and a continuation be $s \in \{0, 1\}^m$, and define

$$T_m(c, s) = \bigoplus_{i=1}^m (c_i \wedge s_i).$$

Every context induces a distinct continuation function: if $c \neq c'$, choose a coordinate j with $c_j \neq c'_j$ and set $s = e_j$. Hence there are 2^m distinct context profiles. Nevertheless T_m has an $O(m)$ -size Boolean circuit. Thus even exponentially many distinct continuation behaviours do not force a large general circuit.

The lesson is stronger than the earlier context-count warning. What would be needed is not merely many contexts or many interactions, but an explicit compact description for which the interaction pattern itself resists all reusable summaries. If an arbitrary Boolean interaction table $R_m(c, s)$ is supplied instead, standard counting produces tables requiring exponential circuit size, but such a table generally has an exponential-size description. The useful lower-bound target therefore lies strictly between these two extremes:

compact description + rich interaction + provable resistance to sharing.

The third term is the unresolved part.

14.6 A multiplexer bridge for contextual memory

The contextual-memory idea suggests a concrete bridge to current constructive lower-bound technology. The multiplexer (storage-access) function has address bits that select among many data bits, so a small change in context can change which data location must be consulted. Carmosino, Dang, and Jackman give an efficient refuter for a nontrivial DeMorgan lower bound for the multiplexer using gate elimination.[21]

The first test of the bridge can be made unconditional and negative. On complete assignments, the semantic extension profile agrees with the original function: $F_\varphi(\alpha) = \varphi(\alpha)$. Therefore any lower bound for MUX obtained by restricting a circuit for its semantic extension to complete assignments pulls back only the MUX lower bound itself; it does not amplify it. In the opposite direction, the semantic extension of MUX admits a polynomial-size circuit by explicitly testing which addresses and data positions remain compatible with a partial assignment. Thus the naive MUX bridge does not create a superpolynomial residual lower bound.

The more informative adversarial test is indirect storage access (ISA), which introduces two stages of contextual selection. In a standard parametrization, an outer address selects a pointer block, the selected pointer selects a data position, and the output is the data bit at that position. Classical Nechiporuk arguments give an explicit near-quadratic formula-size lower bound for ISA, while a general circuit can share the intermediate pointer computation and remain linear-size up to lower-order terms. [12, 13, 14]

This is a particularly useful calibration for the present intuition: contextual interaction can force a large representation in a non-sharing model while remaining compact in a general circuit. The phenomenon is therefore not contextual richness itself, but the inability to share an intermediate semantic summary.

The same point survives semantic extension. Let a be the outer k -bit address, let X_j be the j -th ℓ -bit pointer block, and let $Y = (Y_0, \dots, Y_{2^\ell-1})$ be the data bits. For a partial assignment ρ , let $A_j(\rho)$ assert that the address remains compatible with j , let $P_{j,t}(\rho)$ assert that X_j can still be completed to t , and let $D_t(\rho)$ assert that Y_t can still be set to 1. Then the semantic extension profile satisfies

$$F_{\text{ISA}}(\rho) = 1 \iff \bigvee_j \bigvee_t (A_j(\rho) \wedge P_{j,t}(\rho) \wedge D_t(\rho)) = 1.$$

Because j ranges over at most 2^k pointer blocks and t over at most 2^ℓ data positions, this gives a polynomial-size circuit in the ordinary input length of

the ISA instance. The semantic extension therefore does not manufacture the missing general-circuit hardness even for a two-stage contextual-memory function.

Proposition 14.6.1 (Contextual-memory stress test). *Exponential context-profile multiplicity, multiplexer-style storage access, and two-stage indirect storage access are each compatible with polynomial-size general Boolean circuits. Consequently none of these properties, by itself or by its semantic extension profile, provides the required superpolynomial general-circuit lower bound for the universal residual family.*

Proof. The ordered branching-model lower bound from Section 13 establishes exponential width from distinct context profiles, but the parity calibration above gives a linear general circuit with exponentially many contexts. For MUX, restricting to complete assignments recovers MUX and the semantic extension has a direct polynomial-size realization. For ISA, the explicit compatibility construction above gives a polynomial-size realization of the complete semantic extension profile. Hence none of these contextual phenomena alone can force a superpolynomial general-circuit lower bound. $\square$ $\square$

The correct surviving target is therefore narrower than the original MUX proposal: seek a compact semantic family for which intermediate contextual summaries cannot be shared without destroying correctness. In schematic form, the unresolved implication is

rich contextual interaction $\longrightarrow$ forced non-shareability $\longrightarrow$ general-circuit lower bound.

No theorem for the second arrow is claimed here. The value of the stress test is that it removes several superficially plausible substitutes for that missing theorem.

14.7 The residual refuter conjecture

Conjecture 14.7.1 (Constructive residual lower-bound principle). *There is a polynomial $s(m)$ and a deterministic algorithm $\mathcal{R}$ running in time polynomial in m and the description length of its input circuit such that, for every circuit C of size at most $s(m)$ attempting to compute the universal residual function $\hat{F}_m$, $\mathcal{R}(C, m)$ outputs a valid partial assignment ρ satisfying*

$$C(\eta(\rho)) \neq F_m(\rho).$$

This conjecture is intentionally weaker than $P \neq NP$. It asks for a constructive circuit lower bound at an explicit semantic target. Its plausibility should be tested against the strongest existing general-circuit techniques before any asymptotic claim is made.

14.8 Constructiveness and the original quantifier programme

The refuter perspective also clarifies the earlier algorithm-relative invariant. A certificate $\Gamma(A, x)$ that merely says that A fails has little value unless the failure can be forced by a structural process. A refuter offers a concrete mechanism:

$$A \longrightarrow C_A \longrightarrow \rho \longrightarrow \text{semantic mismatch.}$$

The last step is independently checkable.

This is close in spirit to the original resource-transfer architecture, but it avoids defining the resource tautologically as the running time. The unresolved problem is to prove that a sufficiently powerful refuter exists for the universal residual family.

Residual Meta-Complexity and Magnification

15.1 Exact residual circuit-size testing

The residual family also defines a natural meta-complexity problem.

Definition 15.1.1 (Residual minimum-circuit problem). *Define ResMCSP to be the problem whose input is a Boolean formula φ together with an integer s , asking whether*

$$\text{CC}(\widehat{F}_\varphi) \leq s.$$

The formula φ is a succinct description of the exponentially larger residual truth table. Thus ResMCSP is a genuine succinct circuit-size question, rather than ordinary MCSP with an explicitly listed truth table.

Several variants should be distinguished:

- exact versus promise versions;
- uniform-family versus single-function formulations;
- exact circuit size versus approximate computation on valid partial assignments;
- whole-profile versus fixed-layer residual complexity.

The purpose of this taxonomy is not to assign a complexity class to every variant immediately. It is to identify where standard meta-complexity machinery might attach.

15.2 Magnification as a possible route

Atserias and Müller develop general circuit and formula magnification results in which small lower bounds for carefully constructed approximation problems can imply much stronger lower bounds; their work also gives a uniform magnification result for MCSP. [11]

This suggests the following conditional research architecture:

weak lower bound for a residual approximation problem

$\Downarrow$

magnification theorem

$\Downarrow$

strong circuit lower bound for the residual family.

The missing step is not magnification itself. It is the construction of a residual approximation or promise problem satisfying the hypotheses of a known magnification theorem.

Question 15.2.1 (Residual magnification question). *Can one define a succinct, sufficiently sparse, semantically natural residual promise problem for which a slightly superlinear lower bound is known or attainable, and which fits an existing uniform magnification theorem?*

A positive answer would not automatically prove $P \neq NP$, because the final connection to the universal P-uniform residual family would still have to be established. It would nevertheless turn the vague instruction “find a lower bound” into a sequence of concrete and independently checkable targets.

15.3 Why meta-complexity does not trivialize the problem

The danger is circularity. A meta-problem whose correct answer is simply “no small residual circuit exists” is not a lower-bound certificate.

An acceptable meta-complexity route must provide a structural witness or approximation problem whose hardness is itself independently proved or magnified. This mirrors the earlier audit rule:

definition of desired hardness $\neq$ proof of hardness.

The residual formulation is useful because it gives a compact description of the semantic object. It does not eliminate the need for an independently justified hardness mechanism.

15.4 Uniform versus nonuniform residual hardness

The exact target for $P \neq NP$ is

$$(F_m) \notin \text{P-uniform-SIZE}[m^{O(1)}].$$

A superpolynomial lower bound on ordinary circuit complexity $\text{CC}(\hat{F}_m)$ is stronger and therefore sufficient.

This distinction matters in a meta-complexity setting because a hard instance of ResMCSP can fail to correspond to an efficiently constructible family. Conversely, a P-uniform family can always be viewed nonuniformly.

The research programme should therefore keep three notions separate:

single-function circuit hardness, familywise nonuniform hardness, P-uniform hardness.

Limits, Status, and the Exact Frontier

16.1 Intermediate Concluding Position

The verification-generation intuition is not empty. It points directly at the formal heart of NP: a polynomially bounded certificate can be checked locally, while the existence of some certificate is a global existential statement. For SAT, the relation among verification, decision, and search can be made exact. If one could prove a superpolynomial lower bound against every certifying SAT generator, then $P \neq NP$ would follow immediately.

The present audited programme strengthens the research programme by refusing to treat the missing lower bound as an unnamed mystery. It is now decomposed into identifiable obligations: universal quantification over algorithms, reduction stability, a resource lower bound, and a genuine escape from known lower-bound barriers. This decomposition does not solve the problem, but it makes each future advance or failure mathematically inspectable.

What cannot be done is to skip the universal lower-bound step.

The motivating argument therefore becomes rigorous up to the point where it asks for precisely the theorem that the field has not yet obtained. The gap is not a matter of rhetoric. It is a quantifier gap:

many observed hard searches $\nRightarrow$ no polynomial-time search algorithm exists.

Likewise,

easy verification $\nRightarrow$ hard search,

unless the latter is established by a lower bound.

This should not be interpreted as a defeat of the research idea. It is the point at which a research idea becomes a mathematical program. The next step is to invent or identify a structural invariant for which a universal lower bound can actually be proved.

The standard of this document is therefore intentionally two-sided. We refuse to dismiss a difficult proposal merely because the conclusion is famous or currently open. But we also refuse to manufacture a theorem by allowing philosophical intuitions, empirical difficulty, cryptographic practice, finite experiments, or multiple independent auditors to silently replace an asymptotic complexity lower bound.

A future proof of $P \neq NP$ would have to cross that final gap. The purpose of this volume is to mark the gap exactly, preserve the valid mathematics already obtained, and narrow the next target to a statement that, if proved, would genuinely finish the problem.

Audit Ledger

Claim / object	Status	Reason / boundary
Verifier characterization of NP	Theorem	Standard verifier equivalence.
SAT search-to-decision reconstruction	Theorem	Explicit prefix-extension construction.
Finite auditing remains polynomial-time	Proposition	Closure under fixed composition.
Finite testing cannot settle unrestricted language identity	Proposition	Finiteness versus asymptotic universality.
Gödel incompleteness implies $P \neq NP$	Rejected	No direct complexity implication.
Candidate-space explosion implies an exponential lower bound	Rejected	Algorithms need not enumerate candidates.
Cryptographic security alone implies $P \neq NP$	Rejected	Security assumptions are logically distinct.
Universal SAT-search lower bound	Open core	Would immediately imply $P \neq NP$.
Pointwise universal $\Gamma(x)$	False in general	Destroyed by hard-coding.
Algorithm-relative $\Gamma(A, x)$	Conjectural framework	Needs universal adversarial lower bound.
Semantic extension profile F_φ	Established construction / prior art	Canonical existential extension; closely related to classical monotonisation.
Universal residual family F_m	Definition + theorem	Fixed semantic family containing SAT by restriction.
P-uniform polynomial circuits for F_m	Equivalent to $SAT \in P$	Exact characterization proved; novelty claim limited to the present synthesis.
Monotone-extension equivalence	Theorem	Fixed local encoding translation identifies the profile with the classical monotone-extension viewpoint.
Residual layer decomposition	Theorems / research target	Boundary-layer upper bounds and an intermediate-layer question.
Residual refuter programme	Conjectural / research direction	Constructive lower-bound target inspired by modern refuter theory.

Residual meta-complexity	Research direc- tion	Succinct circuit-size problem attached to the semantic profile.
Superpolynomial general- circuit lower bound for F_m	Open target	Would imply $P \neq NP$.
Lifting route	Research direc- tion	Centralized-to-distributed transfer unre- solved.
Restricted sensitivity / proof lower bounds	Model-specific	No automatic general lower bound.
Semantic witness-extension operator	Definition + the- orem	Generalizes the SAT residual to prefix- self-reducible NP relations.
Residual subfunction profile	Definition + proposition	Classical lower-bound laboratory; unre- stricted lower bound open.
Residual-SAT completeness	Theorem	The residual extension-query language is NP-complete; the novelty target is the complete profile.
Semantic contextual muta- tion	Definition + proposition	Tiny context changes can produce max- imal changes in completion fibres; no general-circuit lower bound follows from the metric alone.
Context profile / instruction memory	Definition + re- stricted theorem	Distinct continuation profiles force width in ordered branching diagrams; transfer to general circuits is open.
Contextual-interaction cali- bration	Negative cali- bration theorem	Rich context profiles can coexist with lin- ear general circuits; the missing property is forced non-shareability.
MUX / ISA stress test	Negative cali- bration theorem	Storage-access and two-stage contextual interaction remain polynomially realiz- able in general circuits; naive contextual amplification is ruled out.
Gate-elimination route	Research direc- tion	Contemporary unrestricted-circuit methodology; transfer to the residual family open.
Prior-art boundary	Established cali- bration	The raw extension operation is not claimed as new; the synthesis and lower- bound architecture remain the subject of study.
Constructive refutation	Research direc- tion	Efficient counterexample production would strengthen a residual lower-bound theorem.
Magnification route	Research direc- tion	Existing magnification machinery may ap- ply only after a suitable residual promise problem is constructed.

Adversarial Pre-Mortem

Quantifier attack

Write every quantifier explicitly. The dangerous form is

$$\forall x \forall A \quad T_A(x) \geq \Gamma(x).$$

The hard-coding theorem shows why a genuine worst-case statement instead takes the form

$$\forall A \exists x \quad T_A(x) \text{ is large.}$$

Representation attack

Apply variable renaming, dummy-variable padding, clause reordering, redundant tautologies, polynomial-time preprocessing, and alternate SAT encodings. A semantic measure should be invariant or polynomially stable.

Algorithmic bypass attack

Try dynamic programming, algebraic computation, compression, symmetry breaking, meet-in-the-middle, inclusion–exclusion, memoization, and representations unlike the motivating search narrative.

Circularity attack

Do not define the desired invariant by the unknown optimum running time or circuit size. A valid lower-bound certificate needs an independently checkable structural content.

Restricted-model attack

If the lower bound is for communication, monotone circuits, bounded depth, decision trees, or a proof system, state the missing transfer theorem to general computation.

Barrier attack

Test relativization, natural-proofs behavior, and algebrization. A semantic label is not a barrier-escape theorem.

Empirical overreach attack

Delete every experiment and ask whether the theorem still follows. Finite experiments can falsify conjectures and suggest proof targets; they do not supply the universal asymptotic quantifier.

Final Mathematical Position

The strongest established result of this revision is the semantic characterization

$$\boxed{P = NP \iff (F_m) \text{ has P-uniform polynomial-size circuits.}}$$

A superpolynomial lower bound for the ordinary (nonuniform) Boolean circuit complexity of $\hat{F}_m$ is a stronger sufficient target: it rules out the P-uniform case, but it is not the only logically possible route to $P \neq NP$.

That stronger lower bound is not proved here. Consequently the book does not claim $P \neq NP$.

The appropriate conclusion is therefore neither victory nor dismissal. The verification-generation intuition has been converted into a finite semantic object, its solver-to-object transfer has been proved, its quantifier vulnerabilities have been tested, and the contextual-memory programme has been stress-tested against parity, multiplexer, and indirect-storage-access calibrations. These tests rule out several naive forms of contextual hardness and localize the remaining obstacle more sharply: a compact semantic family would need a provable obstruction to the sharing of intermediate contextual summaries.

Bibliography

- [1] Clay Mathematics Institute, *P versus NP*, Millennium Prize Problems, accessed September 2026. <https://www.claymath.org/millennium/p-vs-np/>.
- [2] Stephen A. Cook, “The Complexity of Theorem-Proving Procedures,” *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158, doi:10.1145/800157.805047.
- [3] Theodore Baker, John Gill, and Robert Solovay, “Relativizations of the $P = ? NP$ Question,” *SIAM Journal on Computing* 4(4), 1975, pp. 431–442, doi:10.1137/0204037.
- [4] Alexander A. Razborov and Steven Rudich, “Natural Proofs,” *Journal of Computer and System Sciences* 55(1), 1997, pp. 24–35, doi:10.1006/jcss.1997.1494.
- [5] Scott Aaronson and Avi Wigderson, “Algebrization: A New Barrier in Complexity Theory,” *ACM Transactions on Computation Theory* 1(1), 2009, Article 2, doi:10.1145/1490270.1490272.
- [6] Ryan Williams, “Nonuniform ACC Circuit Lower Bounds,” *Proceedings of the IEEE Conference on Computational Complexity*, 2011, pp. 115–125, doi:10.1109/CCC.2011.36.
- [7] Hao Huang, “Induced Subgraphs of Hypercubes and a Proof of the Sensitivity Conjecture,” *Annals of Mathematics* 190(3), 2019, pp. 949–955, doi:10.4007/annals.2019.190.3.6.
- [8] Siddharth Iyer, “Lifting for Arbitrary Gadgets,” ECCC Report TR25-036, 29 March 2025. <https://eccc.weizmann.ac.il/report/2025/036/>.
- [9] Guangxu Yang and Jiapeng Zhang, “Deterministic Lifting Theorems for One-Way Number-on-Forehead Communication,” ECCC Report TR25-073, revision 3, 5 July 2026. <https://eccc.weizmann.ac.il/report/2025/073/>.
- [10] Jan Krajíček, *Bounded Arithmetic, Propositional Logic, and Complexity Theory*, Cambridge University Press.
- [11] Albert Atserias and Moritz Müller, “Simple general magnification of circuit lower bounds,” arXiv:2503.24061, 2025. <https://arxiv.org/abs/2503.24061>.
- [12] E. I. Nechiporuk, “On a Boolean function,” *Doklady Akademii Nauk SSSR* 169(4), 1966, pp. 765–766; English translation, *Soviet Mathematics Doklady* 7(4), 1966, pp. 999–1000.
- [13] Hartmut Klauck, “One-Way Communication Complexity and the Nečiporuk Lower Bound on Formula Size,” *SIAM Journal on Computing* 36(6), 2007, pp. 1495–1510, doi:10.1137/S009753970140004X. <https://doi.org/10.1137/S009753970140004X>.
- [14] John E. Savage, *Models of Computation: Exploring the Power of Computing*, Addison–Wesley, 1998.

- [15] A. V. Chashkin, “Lower bounds for the complexity of restrictions of Boolean functions,” *Discrete Applied Mathematics* 114(1–3), 2001, pp. 61–93, doi:10.1016/S0166-218X(00)00361-9.
- [16] Randal E. Bryant, “Graph-Based Algorithms for Boolean Function Manipulation,” *IEEE Transactions on Computers* 35(8), 1986, pp. 677–691, doi:10.1109/TC.1986.1676819.
- [17] Adnan Darwiche and Pierre Marquis, “A Knowledge Compilation Map,” *Journal of Artificial Intelligence Research* 17, 2002, pp. 229–264, doi:10.1613/jair.989.
- [18] Roberto Sebastiani, “Entailment vs. Verification for Partial-Assignment Satisfiability and Enumeration,” in *Automated Deduction – CADE 30*, Lecture Notes in Computer Science, 2025, pp. 717–735, doi:10.1007/978-3-031-99984-0_37.
- [19] Noel Arteché, Albert Atserias, Susanna F. de Rezende, and Erfan Khaniki, “The Proof Analysis Problem,” in *Proceedings of the 66th Annual IEEE Symposium on Foundations of Computer Science*, 2025, pp. 2555–2576, doi:10.1109/FOCS63196.2025.00133.
- [20] Marco Carmosino, Ngu Dang, and Tim Jackman, “Convergent Gate Elimination and Constructive Circuit Lower Bounds,” arXiv:2602.17942, 2026. <https://arxiv.org/abs/2602.17942>.
- [21] Marco Carmosino, Ngu Dang, and Tim Jackman, “Constructive Separations from Gate Elimination,” arXiv:2604.23958, 2026. <https://arxiv.org/abs/2604.23958>.
- [22] Elizaveta A. Okol’nishnikova, “On Operations of Geometrical Projection and Monotone Extension,” Electronic Colloquium on Computational Complexity, Report TR00-006, 2000. <https://eccc.weizmann.ac.il/eccc-reports/2000/TR00-006/>.
- [23] Matthew Gwynne and Oliver Kullmann, “A framework for good SAT translations, with applications to CNF representations of XOR constraints,” arXiv:1406.7398, 2014. <https://arxiv.org/abs/1406.7398>.
- [24] Lijie Chen, Ce Jin, Rahul Santhanam, and Ryan Williams, “Constructive Separations and Their Consequences,” *TheoretCS* 3, Article 3, 2024, doi:10.46298/theoretcs.24.3.
- [25] Jiayu Li and Tianqi Yang, “ $3.1n-o(n)$ Circuit Lower Bounds for Explicit Functions,” in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, 2022, pp. 1180–1193, doi:10.1145/3519935.3519976.
- [26] Bruno Pasqualotto Cavalar, Mika Göös, Artur Riazanov, Anastasia Sofronova, and Dmitry Sokolov, “Monotone Circuit Complexity of Matching,” ECCC Report TR25-102, 2025. <https://eccc.weizmann.ac.il/report/2025/102/>.
- [27] Marko Chalupa, “An AC^0 Lower Bound for Random Satisfiable 3-CNF under Standard Random Restrictions,” ECCC Report TR25-124, revision 7, 13 August 2025. <https://eccc.weizmann.ac.il/report/2025/124/>.

SEMANTIC EXTENSION PROFILES

The lower-bound frontier is explicit.

This volume develops a concrete semantic extension-profile framework for SAT, calibrates it against prior monotone-extension and SAT-representation work, proves the uniform realization characterization, and identifies constructive, meta-complexity, proof-complexity, lifting, and general-circuit targets.

The book preserves the useful mathematics of the preceding research programme while rejecting hard-coding, representation artifacts, restricted-model overreach, circular invariants, and empirical substitutes for asymptotic proof.

Pearl Bipin Pulickal

Independent Researcher

No claim of a solved P versus NP problem is made in this volume.